BÁO CÁO TỔNG HỢP ĐỒ ÁN

XÂY DỰNG TRÒ CHƠI BI-A HAI NGƯỜI QUA MẠNG

Học phần: Lập trình mạng

Trường / Khoa / Lớp: [Bổ sung]

Giảng viên hướng dẫn: [Bổ sung]

Nhóm thực hiện: [Tên nhóm]

Lộc — [MSSV] · Minh — [MSSV]

Trung — [MSSV] · Mạnh — [MSSV]

Ngày hoàn thiện báo cáo: 28/10/2026 — múi giờ Việt Nam.

Phiên bản mã ứng dụng đối chiếu: 00bf1cc.

MỤC LỤC

PHÂN CÔNG VÀ QUY ƯỚC

Thành viên	Phân hệ phụ trách	Use Cases	Số biểu đồ	Loại biểu đồ
Lộc	Quản lý kết nối & Giao thức truyền tin	UC-C01 → UC-C07 (7 UC)	3	Use Case, Sequence, Activity
Minh	Xác thực & Quản lý dữ liệu	UC-D01 → UC-D07 (7 UC)	4	Use Case, Activity ×2, Sequence
Trung	Gameplay & Engine Vật lý	UC-G01 → UC-G08 (8 UC)	3	Use Case, Activity, Sequence
Mạnh	Giao diện & Cá nhân hóa	UC-UI01 → UC-UI09 (9 UC)	4	Use Case, Activity, Sequence, Component
Tổng	4 phân hệ	31 UC	14	4 Use Case, 5 Activity, 4 Sequence, 1 Component

Use case biểu diễn mục tiêu/chức năng ở ranh giới phân hệ. Với một số xử lý nền, GameRoom hoặc GameScene được dùng làm tác nhân hệ thống đối với phân hệ đang xét. Các bước kỹ thuật của engine được mô hình hóa như use case chức năng theo phạm vi bài tập; không đồng nhất mọi bước nội bộ với một thao tác riêng mà người chơi trực tiếp bấm.

Một chức năng xuyên nhiều phân hệ có các góc nhìn khác nhau: chuyển gói và điều phối phiên; xác thực/ghi dữ liệu; quyết định/mô phỏng gameplay; hiển thị và nhận thao tác. Ví dụ UC-UI08 chọn và xem skin, còn UC-D03 kiểm tra quyền và lưu skin. Hai use case liên quan nhưng không thay thế nhau.

Mỗi biểu đồ Use Case liệt kê đủ use case của phân hệ. Các Activity/Sequence đi sâu vào luồng đại diện được nêu ở chú thích; không có yêu cầu mỗi use case phải có một biểu đồ Sequence riêng. Tổng số biểu đồ trong tài liệu được giữ đúng 3/4/3/4.

DANH MỤC ẢNH MINH HỌA GIAO DIỆN

Phần 1: Mở đầu

1. Lý do chọn đề tài

Trò chơi nhiều người qua mạng kết hợp các vấn đề cốt lõi của lập trình mạng: duy trì kết nối, trao đổi dữ liệu có cấu trúc, quản lý phiên, xử lý đồng thời và phản hồi trạng thái trên giao diện. Với bi-a, người chơi còn cần quan sát hướng gậy, lực đánh, chuyển động của bi và quyền đánh của từng bên. Vì vậy, đề tài vừa có nghiệp vụ tài khoản và dữ liệu, vừa có yêu cầu phối hợp trạng thái theo thời gian.

Nhóm lựa chọn xây dựng trò chơi bi-a hai người bằng Java và JavaFX, sử dụng TCP Socket để truyền dữ liệu nghiệp vụ, UDP để tìm máy chủ trong LAN và PostgreSQL để lưu dữ liệu. Sản phẩm tạo điều kiện áp dụng kiến thức môn Lập trình mạng vào một quy trình hoàn chỉnh: từ kết nối, đăng nhập và tìm đối thủ đến thi đấu, cập nhật kết quả và tương tác bạn bè.

2. Mục tiêu và sản phẩm của nhóm

Mục tiêu chức năng là cung cấp một ứng dụng có tài khoản người chơi, sảnh, ghép trận hoặc thách đấu, bàn bi-a tương tác, luyện tập solo, bạn bè, chat, bảng xếp hạng và cá nhân hóa. Mục tiêu kỹ thuật là tổ chức rõ trách nhiệm của client, server và cơ sở dữ liệu; xây dựng giao thức nhị phân dùng chung; liên kết thao tác giao diện với xử lý nghiệp vụ và phản hồi từ mạng.

Sản phẩm được trình bày trong báo cáo gồm mã nguồn client/server, schema PostgreSQL và báo cáo.

3. Yêu cầu chức năng và chất lượng

Nhóm yêu cầu	Nội dung	Thành phần liên quan
Kết nối và phiên	Kết nối TCP, khám phá LAN, trao đổi packet, RTT, ghép trận, lời mời và ngắt kết nối	Kết nối và giao thức
Tài khoản và dữ liệu	Đăng ký, đăng nhập, hồ sơ, kết quả, xếp hạng, bạn bè và tin nhắn	Xác thực và dữ liệu
Thi đấu	Khởi tạo bàn, đặt bi, ngắm/lực/xoáy, mô phỏng, luật lượt và solo	Gameplay và vật lý
Tương tác	Màn hình, form, sảnh, chat, lời mời, HUD, kết quả và tùy chọn cá nhân	Giao diện và cá nhân hóa
Tính nhất quán	Hai phía diễn giải cùng một payload; quyền lượt và chuyển trạng thái không mâu thuẫn	Giao thức và trạng thái trận

Nhóm yêu cầu	Nội dung	Thành phần liên quan
Khả năng sử dụng	Hiển thị phản hồi rõ, thao tác trên luồng JavaFX đúng quy tắc, có đường xử lý lỗi	Toàn hệ thống
Khả năng bảo trì	Mã giao thức, nghiệp vụ, dữ liệu và giao diện có trách nhiệm xác định	Toàn hệ thống

Phần 2: Cơ sở lý thuyết và công nghệ sử dụng

1. Mô hình client–server và Socket

Trong mô hình client–server, client chủ động mở kết nối và gửi yêu cầu; server lắng nghe, tạo ngữ cảnh xử lý cho kết nối và trả kết quả. Ở dự án này, ServerMain tiếp nhận socket, ClientHandler xử lý luồng dữ liệu của từng client, LobbyManager quản lý ghép trận và GameRoom quản lý một trận. Cách tổ chức này tách vòng đời kết nối khỏi vòng đời trận đấu: một người đã đăng nhập có thể ở sảnh mà chưa thuộc phòng nào.

TCP cung cấp luồng byte có thứ tự và cơ chế truyền lại ở tầng vận chuyển. Tuy nhiên, một lần ghi không tương ứng một lần đọc. Một packet nghiệp vụ có thể đến qua nhiều lần đọc hoặc nhiều packet có thể nằm liền trong bộ đệm. Do đó, phía nhận cần đọc đủ từng trường theo định dạng chung; không được dùng số byte đang sẵn có như độ dài cố định của toàn bộ thông điệp.

TCP cũng không bảo đảm hai giao diện đang ở cùng trạng thái ứng dụng. Hai lệnh đánh hợp lệ về mặt byte vẫn có thể bị áp dụng lặp nếu ứng dụng không nhận diện lệnh đã xử lý. Tương tự, truyền chính xác thông số cú đánh không tự bảo đảm hai engine kết thúc tại cùng tọa độ. Đây là lý do cần phân biệt độ tin cậy của vận chuyển với tính nhất quán của trò chơi.

UDP phù hợp cho truy vấn khám phá trong LAN vì client có thể phát thông điệp tìm kiếm mà chưa biết địa chỉ server. Dự án dùng UDP 12346 cho BILLIARDS_DISCOVER và phản hồi BILLIARDS_SERVER:12345. Đường khám phá không thay thế kết nối TCP cho tài khoản hoặc thi đấu; khi broadcast không đi qua cấu hình mạng, người chơi vẫn cần nhập IP server.

2. Giao thức nhị phân và tuần tự hóa

Giao thức nhị phân biểu diễn dữ liệu bằng các kiểu byte, boolean, int, long và double theo quy ước. Dự án đặt mã loại một byte trước payload; thứ tự trường phụ thuộc PacketType. Số nhiều byte dùng big-endian. Chuỗi gồm độ dài kiểu int rồi dữ liệu UTF-8; bộ đọc giới hạn độ dài để không cấp phát theo giá trị tùy ý

Ưu điểm của cách này là payload nhỏ và đọc/ghi trực tiếp bằng Java. Đổi lại, hai đầu phải thống nhất chính xác kiểu, thứ tự và số trường. Nếu server chỉ ghi boolean nhưng client đọc thêm string, các byte kế tiếp có thể bị hiểu sai hoặc quá trình đọc bị chặn. Lỗi SETTINGS_RESPONSE trong phần kiểm thử là ví dụ cụ thể của việc hợp đồng hai đầu chưa khớp.

3. Luồng xử lý và JavaFX

Server xử lý nhiều kết nối nên cần quản lý dữ liệu dùng chung như danh sách online, hàng chờ và trạng thái phòng. Đồng bộ hóa một phương thức giúp bảo vệ vùng thao tác cụ thể, nhưng không tự biến chuỗi tác vụ liên quan nhiều đối tượng hoặc nhiều câu SQL thành một giao dịch nguyên tử.

Client có luồng nhận mạng và JavaFX Application Thread. Luồng nhận phân tích packet, còn thay đổi node và các thao tác giao diện phải chuyển về luồng JavaFX, thường qua Platform.runLater. Tác vụ đọc socket không nên chạy trực tiếp trên luồng hiển thị vì có thể chặn thao tác người dùng. AnimationTimer hỗ trợ cập nhật hình theo khung hình; nó không đồng nghĩa mỗi callback cách nhau đúng một khoảng thời gian cố định.

4. Nguyên lý mô phỏng bi và làm mượt hiển thị

Mỗi bi được mô hình hóa bằng vị trí, vận tốc, trạng thái hoạt động và tham số xoáy. Engine lần lượt cập nhật chuyển động, tác động ma sát, kiểm tra va chạm bi–bi, phản xạ băng và phát hiện lỗ. Việc chia một lần update thành các bước con giúp giảm khoảng dịch chuyển giữa hai lần kiểm tra va chạm. Tuy nhiên, nó không thay thế bộ bước thời gian cố định hoặc cơ chế hiệu chỉnh trạng thái giữa hai máy.

Các hệ số ma sát và phục hồi trong dự án là tham số của mô hình trò chơi; không được xem là số liệu vật lý thực nghiệm. Hướng gậy nhận qua mạng được nội suy để giảm cảm giác giật khi các mẫu tới cách nhau. Nội suy chỉ làm mượt cách vẽ giữa các trạng thái đã nhận, không xác nhận cú đánh hoặc sửa khác biệt vị trí bi cuối lượt.

5. PostgreSQL, JDBC và bảo vệ dữ liệu

Cơ sở dữ liệu quan hệ liên kết hồ sơ, kết quả trận, bạn bè và tin nhắn bằng khóa chính/khóa ngoại. JDBC cung cấp kết nối, PreparedStatement và ResultSet để mã Java thực hiện truy vấn. PreparedStatement tách giá trị đầu vào khỏi cấu trúc câu SQL; các quy tắc nghiệp vụ như quyền thay skin hoặc quyền ghi kết quả vẫn phải được kiểm tra ở ứng dụng.

Một giao dịch dữ liệu cần bảo đảm các bước phụ thuộc nhau cùng thành công hoặc cùng bị hủy. Ví dụ ghi lịch sử và tăng EXP nên được xem xét như một đơn vị nghiệp vụ. Phần hiện trạng nêu rõ recordMatchResult chưa có transaction bao trọn các cập nhật. Index có thể hỗ trợ truy vấn phù hợp nhưng hiệu quả còn phụ thuộc điều kiện truy vấn, dữ liệu và kế hoạch thực thi; báo cáo không gán một độ phức tạp cố định cho mọi truy vấn.

Mật khẩu trong dự án được băm với SHA-256 và salt ngẫu nhiên. Salt giúp tránh việc các mật khẩu giống nhau luôn cho cùng giá trị lưu, nhưng băm nhanh chưa phải lựa chọn tối ưu để chống dò mật khẩu. TLS cho đường truyền và thuật toán chuyên dụng như Argon2id hoặc bcrypt là hướng hoàn thiện,.

Phần 3: Kiến trúc và thiết kế

1. Các thành phần và đường đi dữ liệu

Thành phần	Trách nhiệm	Liên kết chính
Client JavaFX	Điều hướng, form, bàn chơi, mô phỏng và âm thanh	Gửi/nhận qua NetworkClient
NetworkClient và protocol	Kết nối, đọc/ghi payload, callback và đo RTT	TCP đến ClientHandler; UDP discovery
ServerMain và ClientHandler	Nhận kết nối, quản lý phiên và điều phối yêu cầu	AuthManager, LobbyManager, GameRoom, DatabaseManager
LobbyManager và GameRoom	Ghép cặp, quản lý phòng, quyền lượt và kết quả	Hai ClientHandler của trận
AuthManager và DatabaseManager	Xác thực, hồ sơ và lưu trữ	PostgreSQL qua JDBC
PostgreSQL	players, match_history, friendships, messages	Máy chủ ứng dụng truy cập; client không kết nối DB trực tiếp

Một thao tác đăng nhập bắt đầu từ LoginScene, được NetworkClient tuần tự hóa, ClientHandler phân phối đến phần xác thực, DatabaseManager truy vấn tài khoản và kết quả được trả lại client. Giao diện chỉ chuyển cảnh khi xử lý phản hồi phù hợp. Với cú đánh, đường đi khác: GameScene vừa gửi thông số vừa mô phỏng tại máy đánh; GameRoom chuyển thông số sang đối thủ và xử lý báo cáo cuối lượt.

2. Hợp đồng liên kết bốn phân hệ

Điểm bàn giao	Bên cung cấp → bên sử dụng	Điều cần thống nhất
Dữ liệu tài khoản	Dữ liệu hồ sơ → giao diện, qua kết nối mạng	Trạng thái thành công/thất bại và các trường hồ sơ
Tạo trận	Thông tin phòng → bàn chơi và màn hình trước trận	Đối thủ, seed, thông tin trước trận và trạng thái sẵn sàng
Ngắm và đánh	Thao tác đánh → truyền thông → mô phỏng và hiển thị ở đối thủ	Góc, lực, xoáy, tọa độ bi cái và quyền lượt
Kết quả	Kết quả trận → lưu trữ → màn hình kết quả	Người thắng/thua, EXP và thông báo kết thúc
Cá nhân hóa	Thao tác giao diện → yêu cầu mạng → xử lý dữ liệu	Avatar/skin, quyền theo level và phản hồi lưu
Bạn bè và chat	Thao tác giao diện → yêu cầu mạng → xử lý dữ liệu	Định danh đối tượng, dữ liệu hiển thị và vòng đời yêu cầu

Ma trận mô tả đường đi của dữ liệu giữa các thành phần trong hệ thống. Một lớp có thể tham gia nhiều luồng xử lý; chẳng hạn GameScene vừa trình bày HUD vừa gọi engine vật lý. Sự liên kết được xác định theo chức năng và trạng thái nghiệp vụ để các thao tác từ giao diện đến server và cơ sở dữ liệu được xử lý nhất quán.

3. Thiết kế dữ liệu và tính toàn vẹn

players là bảng trung tâm, có định danh tài khoản, email, username, mật khẩu băm, EXP, level, thắng/thua và tùy chọn avatar/gậy. match_history tham chiếu hai người thắng/thua. friendships dùng cặp user_id/friend_id làm khóa; messages có sender_id và receiver_id tham chiếu players. Những liên kết này hỗ trợ truy vấn dữ liệu theo người chơi và tránh tham chiếu tới tài khoản không tồn tại khi ràng buộc được thực thi.

Các ràng buộc hiện có chưa thay thế toàn bộ quy tắc nghiệp vụ. Cột status của friendships là varchar, không có CHECK giới hạn tập trạng thái trong schema. UNIQUE của username gốc cũng không tự tạo tính duy nhất theo LOWER(username). Nếu ứng dụng tra cứu không phân biệt hoa/thường thì cần xem xét ràng buộc tương ứng và tình huống hai yêu cầu đồng thời. Đây là điểm cần cải thiện của thiết kế lưu trữ, không phải kết quả một bài thử tải đã hoàn tất.

4. Các gói tiêu biểu và trạng thái phiên

Packet/luồng	Payload hoặc vai trò	Liên hệ
PING	long timestamp; phản hồi giữ timestamp để tính RTT	Đo độ trễ và hiển thị trên HUD
CUE_AIM	boolean và 4 double; 34 byte kể cả mã loại	Ngắm, lực tương đối và xoáy
CUE_SHOT	6 double; 49 byte kể cả mã loại	Cú đánh của người có lượt
GAME_STATE	Chuyển thông số cú đánh sang đối thủ	Mô phỏng cục bộ của đối thủ
PHYSICS_DONE	keepTurn và isFoul	Báo kết thúc mô phỏng để đổi/giữ lượt
SETTINGS_UPDATE / RESPONSE	Yêu cầu lưu tùy chọn và trạng thái trả về	Lưu tùy chọn và cập nhật giao diện; còn sai lệch parser

Giao thức hiện có 47 mã loại. Các trạng thái nghiệp vụ có thể đọc theo chuỗi: đã kết nối, đã xác thực, đang ở sảnh, đang chờ ghép/lời mời, đã vào phòng, chờ sẵn sàng, thi đấu và kết thúc. Đây là cách diễn giải vòng đời để phân tích; mã hiện tại chưa có một máy trạng thái thống nhất bảo vệ mọi chuyển tiếp. Các tình huống READY lặp hoặc phản hồi lời mời sai tên cho thấy cần kiểm tra điều kiện chuyển trạng thái ở server.

5. Ranh giới triển khai và kiểm chứng

Server chưa có handshake thương lượng phiên bản, token lượt, sequence number cho cú đánh hoặc snapshot/ACK để chốt toàn bộ bàn. Client gửi mẫu ngắm theo ngưỡng thời gian và mức thay đổi; các đường gửi cưỡng bức có thể vượt quy tắc hạn tần thông thường. Do đó, không suy luận rằng mọi luồng luôn bị giới hạn cứng ở 30 mẫu/giây.

Bốn phần tiếp theo trình bày chi tiết đúng phạm vi của từng TV. Phần triển khai sau đó ghép các chức năng thành quy trình sử dụng chung; phần kiểm thử đối chiếu những gì đã chạy và những gì còn cần kiểm tra trước khi nghiệm thu.

Phần 4. QUẢN LÝ KẾT NỐI & GIAO THỨC TRUYỀN TIN

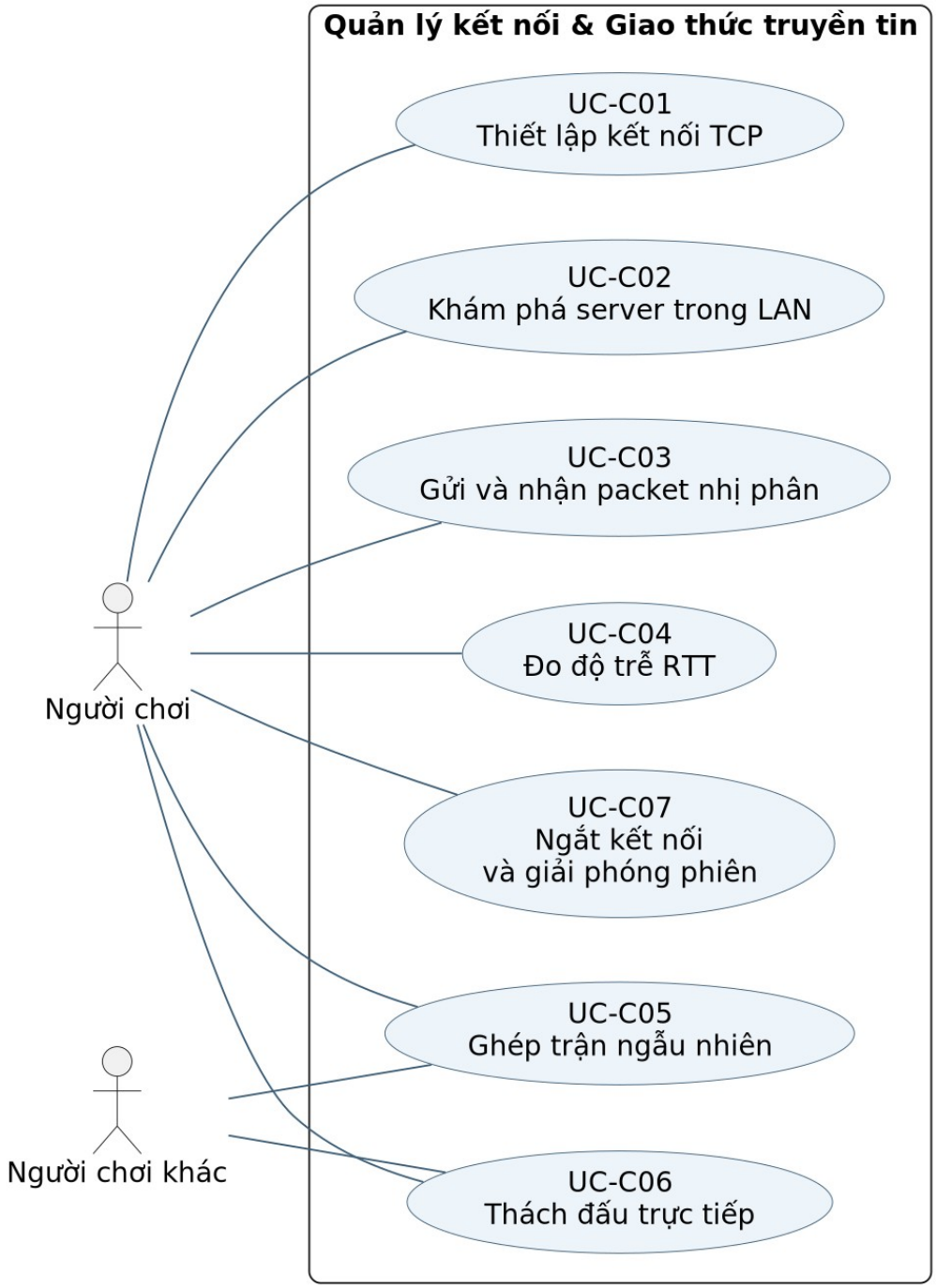
1. Phạm vi và ranh giới trách nhiệm

Phân hệ kết nối và giao thức đảm nhiệm truyền dữ liệu giữa client và server, khám phá máy chủ, định dạng packet, đo độ trễ, ghép trận và quản lý vòng đời kết nối. Các lớp chính là NetworkClient, Packet, PacketType, ServerMain, ClientHandler và LobbyManager. Yêu cầu nhận được được chuyển tới phần xác thực hoặc xử lý nghiệp vụ; kết quả được gửi lại để giao diện cập nhật.

Giao thức có một byte mã loại và các trường tuần tự big-endian. Chuỗi dùng int độ dài cùng UTF-8, giới hạn đọc 65.536 byte. TCP chỉ bảo toàn luồng byte và thứ tự, nên bên nhận phải đọc chính xác payload. Client hiện ghi/flush trực tiếp; chưa có hàng đợi ghi giới hạn riêng hoặc handshake xác nhận phiên bản.

CUE_AIM mang một boolean và bốn double, tổng 34 byte cả mã gói. CUE_SHOT mang sáu double, tổng 49 byte. PING có long timestamp, gửi ngay lúc kết nối rồi mỗi hai giây. Các con số này mô tả định dạng/lịch gửi trong mã, không phải kết quả đo băng thông Internet.

2. Danh mục use case



Mã	Tên use case	Tác nhân chính
UC-C01	Thiết lập kết nối TCP	Người chơi
UC-C02	Khám phá server trong LAN	Người chơi
UC-C03	Gửi và nhận packet nhị phân	Người chơi thông qua chức năng ứng dụng

Mã	Tên use case	Tác nhân chính
UC-C04	Đo độ trễ RTT	Người chơi
UC-C05	Ghép trận ngẫu nhiên	Người chơi và đối thủ
UC-C06	Thách đấu trực tiếp	Người mời và người nhận
UC-C07	Ngắt kết nối và giải phóng phiên	Người chơi, hoặc kết nối bị gián đoạn

3. Đặc tả use case

UC-C01 — Thiết lập kết nối TCP

Tác nhân: Người chơi.

Tiền điều kiện: Đã có địa chỉ/cổng server và mạng khả dụng.

Luồng chính:

1. Nhập hoặc nạp địa chỉ.
2. Mở socket với timeout kết nối 2,5 giây.
3. Tạo stream.
4. Khởi động luồng nhận và ping.
5. Lưu địa chỉ.

Ngoại lệ: Không kết nối được: báo lỗi; người chơi sửa cấu hình hoặc thử lại. Chưa có handshake phiên bản.

Hậu điều kiện: Có kết nối TCP để thực hiện yêu cầu nghiệp vụ.

UC-C02 — Khám phá server trong LAN

Tác nhân: Người chơi.

Tiền điều kiện: Client và server cùng vùng mạng hỗ trợ discovery.

Luồng chính:

1. Gửi BILLIARDS_DISCOVER qua UDP 12346.
2. Nhận BILLIARDS_SERVER:12345.
3. Sử dụng địa chỉ nhận để kết nối TCP.

Ngoại lệ: Không có phản hồi: người chơi nhập địa chỉ thủ công; discovery không bảo đảm tìm server qua Internet.

Hậu điều kiện: Xác định được địa chỉ server hoặc quay về nhập thủ công.

UC-C03 — Gửi và nhận packet nhị phân

Tác nhân: Người chơi thông qua chức năng ứng dụng.

Tiền điều kiện: Socket đang mở; hai phía dùng định dạng tương thích.

Luồng chính:

1. Ghi byte PacketType và các trường.
2. Flush.
3. Phía nhận đọc trọn payload theo loại rồi phân phối tới handler/callback.

Ngoại lệ: Payload sai hoặc stream kết thúc có thể gây IOException; SETTINGS_RESPONSE hiện có lệch định dạng được ghi ở phụ lục kiểm chứng.

Hậu điều kiện: Yêu cầu được chuyển đến đúng bộ phận khi đọc thành công.

UC-C04 — Đo độ trễ RTT

Tác nhân: Người chơi.

Tiền điều kiện: Socket đang hoạt động.

Luồng chính:

1. Client gửi PING kèm long timestamp ngay khi kết nối và định kỳ hai giây.
2. Server trả timestamp.
3. Client tính chênh lệch để hiển thị.

Ngoại lệ: Không nhận lại ping: không có số đo mới; đây chưa phải chính sách timeout đọc 20 giây.

Hậu điều kiện: Có giá trị RTT phục vụ hiển thị, không thay đổi luật chơi.

UC-C05 — Ghép trận ngẫu nhiên

Tác nhân: Người chơi và đối thủ.

Tiền điều kiện: Đã đăng nhập; chưa có phòng.

Luồng chính:

1. Gửi FIND_MATCH.
2. Đưa vào queue nếu chưa có.
3. Khi đủ hai người, tạo GameRoom và gửi MATCH_FOUND cho cả hai.

Ngoại lệ: CANCEL_FIND xóa khỏi queue; người đã có phòng hoặc đã nằm trong queue không được thêm lặp.

Hậu điều kiện: Ghép được phòng hai người hoặc tiếp tục chờ/hủy tìm.

UC-C06 — Thách đấu trực tiếp

Tác nhân: Người mời và người nhận.

Tiền điều kiện: Người mời đăng nhập; có tên người nhận online.

Luồng chính:

1. Gửi tên người nhận.
2. Server trim và tra không phân biệt hoa thường.
3. Phát lời mời.
4. Nhận accept.
5. Xóa queue và tạo phòng khi hợp lệ.

Ngoại lệ: Tên rỗng/tự mời/offline/bận hoặc từ chối/hết hạn đều thất bại; client đếm 10 giây, server chờ 15 giây.

Hậu điều kiện: Có MATCH_FOUND hoặc thông báo không tạo trận; trường hợp phản hồi sai tên còn lỗi vòng đời timer.

UC-C07 — Ngắt kết nối và giải phóng phiên

Tác nhân: Người chơi, hoặc kết nối bị gián đoạn.

Tiền điều kiện: Có socket hoặc phiên đang hoạt động.

Luồng chính:

1. Người chơi ngắt hoặc server phát hiện kết thúc stream.
2. Xóa online/queue.
3. Hủy timer lời mời.
4. Xử lý phòng và đóng socket.

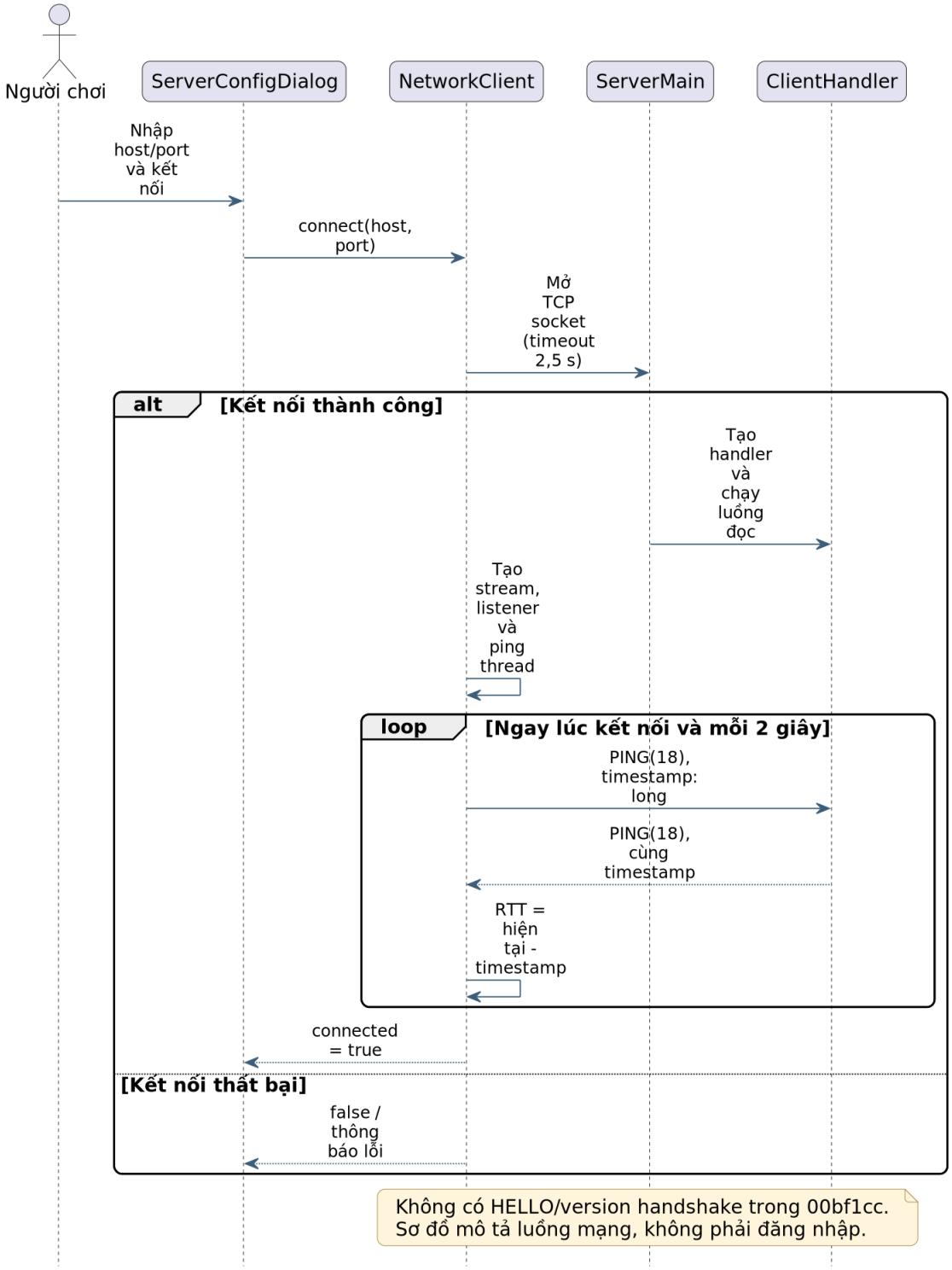
Ngoại lệ: Ngắt trong phòng: người còn lại thắng theo quy tắc hiện có; không có phục hồi ván cũ sau reconnect.

Hậu điều kiện: Phiên được dọn theo logic cleanup; không khẳng định mọi lỗi mạng được phát hiện ngay.

4. Biểu đồ của phân hệ

Sơ đồ tuần tự

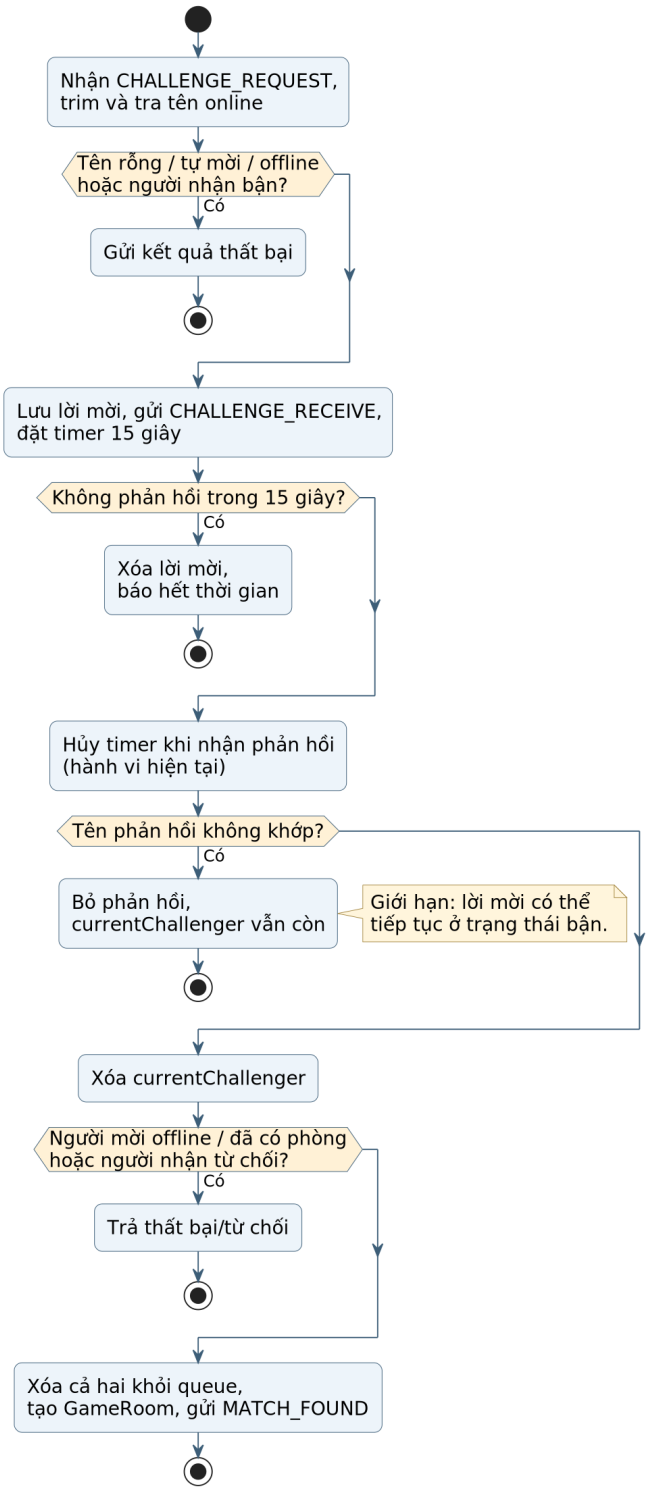
Phạm vi: UC-C01, UC-C03, UC-C04.



Hình 1.2 — Kết nối TCP và trao đổi PING. Loại: Sequence; dựng từ mã nguồn 00bf1cc.

Activity

Phạm vi: UC-C06, UC-C07.



Hình 1.3 — Điều phối thách đấu trực tiếp. Loại: Activity; dựng từ mã nguồn 00bf1cc.

5. Liên hệ triển khai và giới hạn

Sơ đồ Sequence tập trung thiết lập TCP và ping, còn Activity tập trung lời mời thách đấu. Các use case khác vẫn được thể hiện trong Use Case và đặc tả riêng. Timer

lời mời server là 15 giây; hộp thoại client đếm 10 giây. Nhánh phản hồi sai tên trong Activity được giữ lại vì đó là hành vi hiện có, không mô tả như đã sửa.

Sau khi thiết lập kết nối (UC-C01), ứng dụng có thể xử lý đăng ký hoặc đăng nhập (UC-D01/UC-D02). Ghép trận và thách đấu (UC-C05/UC-C06) tạo ngữ cảnh khởi tạo bàn (UC-G01), còn UC-C03 vận chuyển dữ liệu giữa các thành phần. Ngắt kết nối (UC-C07) có thể kích hoạt ghi kết quả (UC-D04) theo trạng thái phòng hiện tại.

PHẦN 2. XÁC THỰC & QUẢN LÝ DỮ LIỆU

Phạm vi chức năng: 7 use case, 4 biểu đồ.

2.1. Phạm vi và ranh giới trách nhiệm

Phân hệ xác thực và dữ liệu kiểm tra tài khoản, quyền cập nhật hồ sơ và thực hiện lưu trữ qua JDBC. AuthManager băm mật khẩu SHA-256 cùng salt ngẫu nhiên 16 byte. DatabaseManager thao tác các bảng players, match_history, friendships và messages trong PostgreSQL. Kết quả xử lý được trả về client để giao diện hiển thị trạng thái phù hợp.

Email được trim và chuyển thường; tra cứu đăng nhập dùng LOWER(username) hoặc LOWER(email). Mã hiện tại chưa kiểm tra định dạng email ở server tương đương client, chưa xác minh quyền sở hữu email bằng thư/OTP và chưa có TLS. Đây là mô tả giới hạn thực tế, không phải tính năng bảo mật đã triển khai.

EXP thắng trận là 100; level = floor(EXP/200)+1. DatabaseManager giữ một Connection chung và recordMatchResult chưa bao các câu SQL trong transaction hoàn chỉnh. Activity 2.3 thể hiện ghi tuần tự và ghi chú việc không tự rollback toàn bộ khi lỗi giữa chừng.

2.2. Danh mục use case

Mã	Tên use case	Tác nhân chính
UC-D01	Đăng ký tài khoản	Người chơi mới
UC-D02	Đăng nhập bằng tên hoặc email	Người chơi
UC-D03	Lưu avatar và skin gậy	Người chơi đã đăng nhập
UC-D04	Ghi kết quả và cập nhật EXP/level	GameRoom (tác nhân hệ thống)
UC-D05	Truy vấn bảng xếp hạng	Người chơi
UC-D06	Quản lý quan hệ bạn bè	Người chơi và người được mời kết bạn
UC-D07	Lưu và truy vấn tin nhắn	Người gửi và người nhận

2.3. Đặc tả use case

UC-D01 — Đăng ký tài khoản

Tác nhân: Người chơi mới.

Tiền điều kiện: Kết nối server và PostgreSQL khả dụng.

Luồng chính:

1. Nhận email, username, mật khẩu.
2. Kiểm tra cơ bản.
3. Băm cùng salt.
4. Trim/chuyển thường email.
5. Kiểm tra trùng và INSERT players.

Ngoại lệ: Email/username trùng hoặc mật khẩu quá ngắn bị từ chối; server chưa kiểm tra định dạng email tương đương client.

Hậu điều kiện: Tạo tài khoản với mật khẩu băm hoặc trả mã lỗi.

Mã nguồn liên quan: AuthManager.register/hashPassword; DatabaseManager.register.

UC-D02 — Đăng nhập bằng tên hoặc email

Tác nhân: Người chơi.

Tiền điều kiện: Tài khoản đã tồn tại; server kết nối DB.

Luồng chính:

1. Tra LOWER(username) hoặc LOWER(email).
2. Xác minh hash với salt.
3. Tạo thông tin phiên và trả hồ sơ.

Ngoại lệ: Sai mật khẩu/không tìm thấy: trả thất bại; định danh trùng chéo username–email chưa được giải quyết tường minh.

Hậu điều kiện: Có phiên xác thực khi thành công.

Mã nguồn liên quan: AuthManager.login/verifyPassword; DatabaseManager.getPlayerByUsername; ClientHandler.handleLogin.

UC-D03 — Lưu avatar và skin gậy

Tác nhân: Người chơi đã đăng nhập.

Tiền điều kiện: Có phiên và lựa chọn tùy chỉnh.

Luồng chính:

1. Nhận avatarId, cueColor/skinId.
2. Kiểm tra điều kiện cấp độ bằng CueSkinConfig.
3. Cập nhật DB và thông tin phiên.
4. Trả boolean.

Ngoại lệ: Không đủ cấp hoặc lỗi DB: trả false; client đang đọc thừa string trong SETTINGS_RESPONSE nên phản hồi có thể không được xử lý đúng.

Hậu điều kiện: DB/phiên cập nhật khi thành công; không khẳng định UI đã xác nhận được.

Mã nguồn liên quan: ClientHandler.handleSettingsUpdate; CueSkinConfig; DatabaseManager.updateSettings.

UC-D04 — Ghi kết quả và cập nhật EXP/level

Tác nhân: GameRoom (tác nhân hệ thống).

Tiền điều kiện: GameRoom đã chọn người thắng/thua.

Luồng chính:

1. Đặt gameFinished.
2. Cộng thắng và 100 EXP.
3. Tính level=floor(EXP/200)+1.
4. Tăng thua.
5. INSERT match_history.
6. Nạp lại hồ sơ.

Ngoại lệ: DB lỗi giữa các câu lệnh có thể để cập nhật dở dang vì chưa có transaction bao trọn.

Hậu điều kiện: Kết quả được ghi theo các câu lệnh thành công; cờ phòng hạn chế lặp trong phiên.

Mã nguồn liên quan: GameRoom.endGame/handleDisconnect; DatabaseManager.recordMatchResult.

UC-D05 — Truy vấn bảng xếp hạng

Tác nhân: Người chơi.

Tiền điều kiện: Có dữ liệu players và kết nối DB.

Luồng chính:

1. Nhận LEADERBOARD_REQUEST.
2. Lấy người chơi theo exp giảm dần với giới hạn.
3. Đóng gói chuỗi và trả client.

Ngoại lệ: Lỗi DB hoặc chưa có dữ liệu: xử lý kết quả trống theo mã hiện tại; chưa kiểm thử tải lớn.

Hậu điều kiện: Client nhận dữ liệu xếp hạng để dựng giao diện.

Mã nguồn liên quan: DatabaseManager.getLeaderboard; ClientHandler.handleLeaderboard.

UC-D06 — Quản lý quan hệ bạn bè

Tác nhân: Người chơi và người được mời kết bạn.

Tiền điều kiện: Tài khoản hợp lệ; dữ liệu người nhận tồn tại.

Luồng chính:

1. Tìm người chơi, tạo lời mời.
2. Chấp nhận/từ chối.
3. Đọc danh sách.
4. Xóa quan hệ theo yêu cầu.

Ngoại lệ: Người nhận không tồn tại hoặc quan hệ không phù hợp: trả trạng thái thất bại; status chưa có CHECK enum trong schema.

Hậu điều kiện: Bảng friendships phản ánh thao tác được chấp nhận.

Mã nguồn liên quan: DatabaseManager: friendship methods; ClientHandler: friend handlers; schema.sql.

UC-D07 — Lưu và truy vấn tin nhắn

Tác nhân: Người gửi và người nhận.

Tiền điều kiện: Hai tài khoản được xác định theo quy tắc hiện tại.

Luồng chính:

1. Lưu tin vào messages.
2. Chuyển thông báo tới client.
3. Truy vấn lịch sử giới hạn và trả chuỗi dữ liệu.

Ngoại lệ: Lịch sử truyền chưa có conversation ID/message ID phục vụ UI; không suy ra mọi phản hồi muộn được phân biệt.

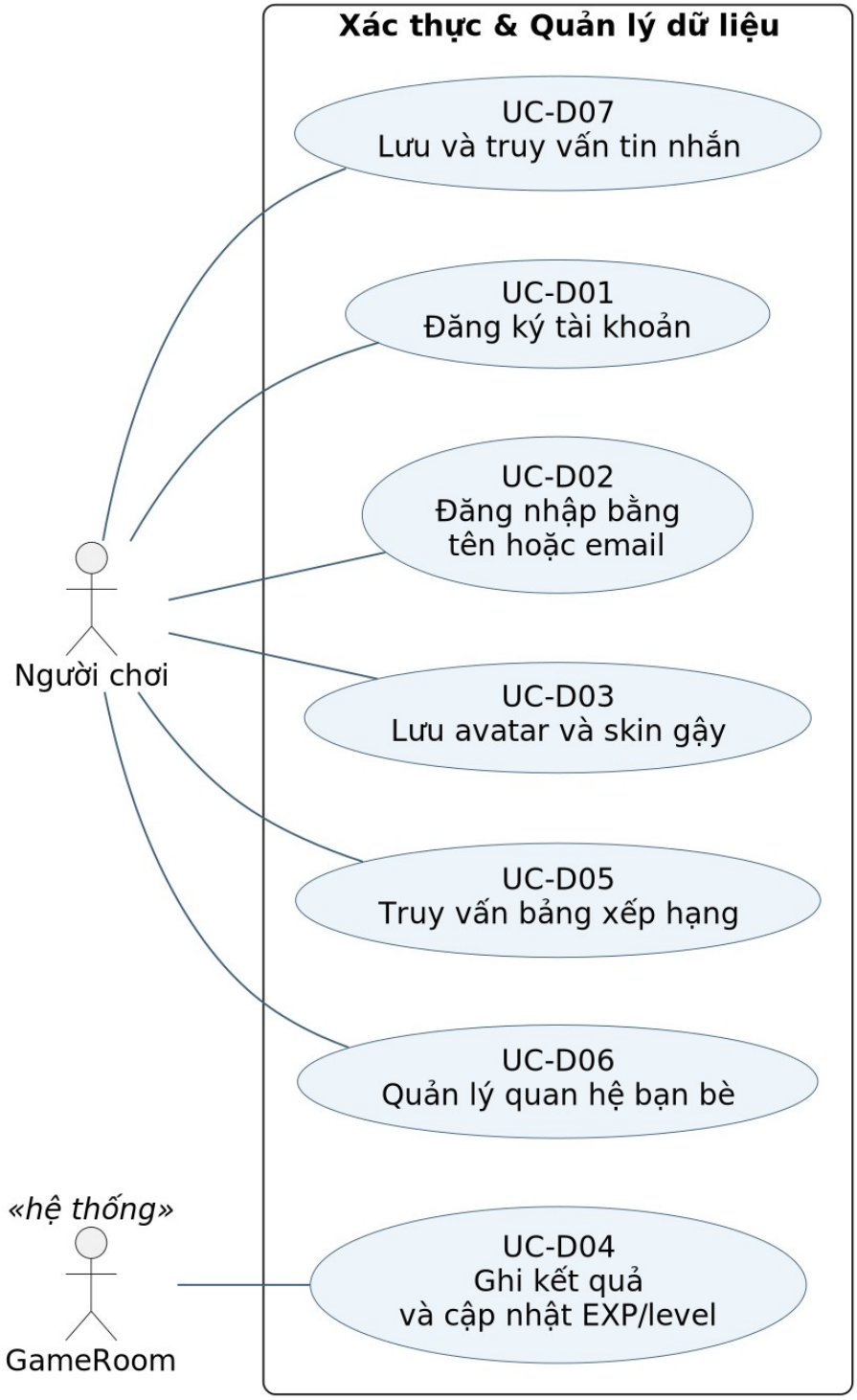
Hậu điều kiện: Tin nhắn được lưu hoặc lịch sử được trả theo khả năng DB.

Mã nguồn liên quan: DatabaseManager.getChatHistory và thao tác messages; ClientHandler.handleChatMessage/handleChatHistory.

2.4. Biểu đồ UML của phân hệ

Biểu đồ 2.1 — Use Case

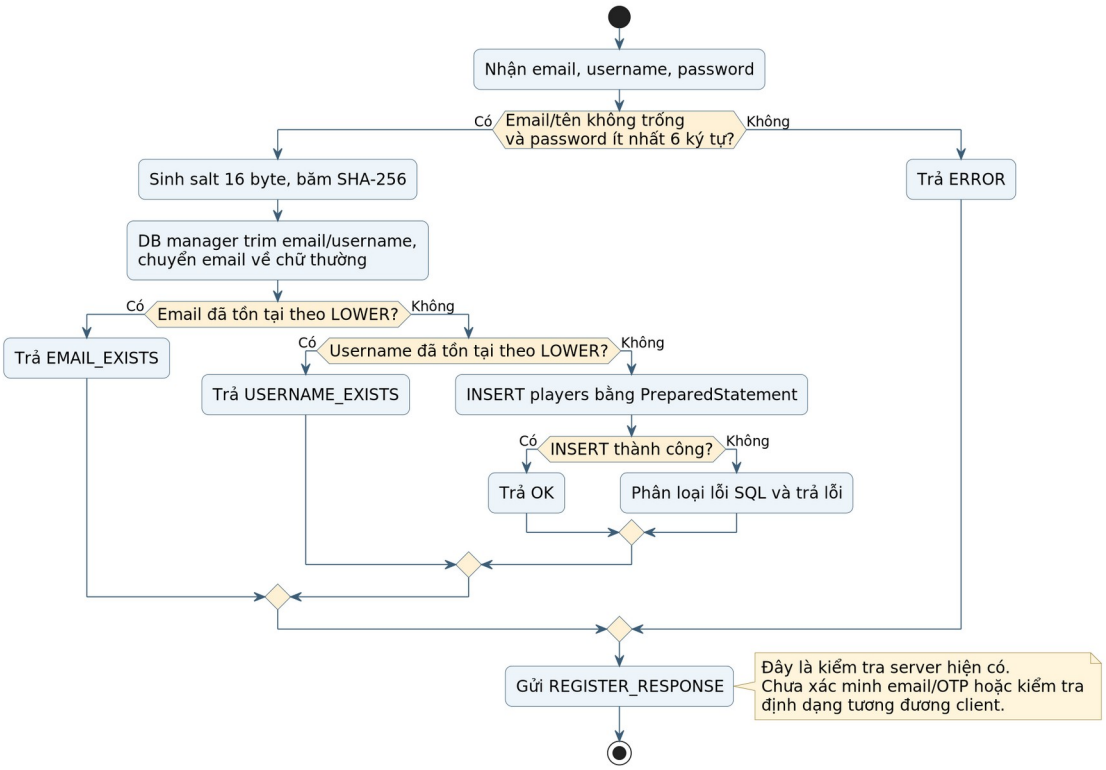
Phạm vi: UC-D01, UC-D02, UC-D03, UC-D04, UC-D05, UC-D06, UC-D07.



Hình 2.1 — Các use case thuộc phân hệ. Loại: Use Case; dựng từ mã nguồn 00bf1cc.

Biểu đồ 2.2 — Activity

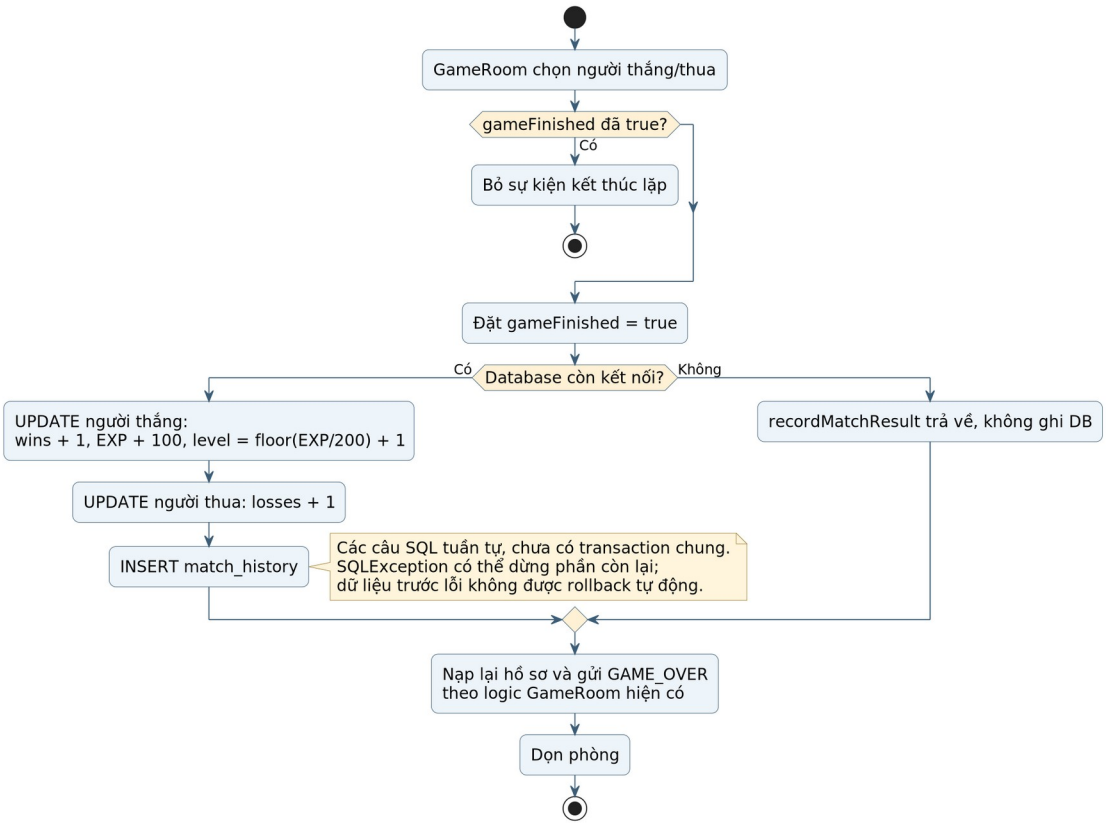
Phạm vi: UC-D01.



Hình 2.2 — Đăng ký và lưu tài khoản. Loại: Activity; dựng từ mã nguồn 00bf1cc.

Biểu đồ 2.3 — Activity

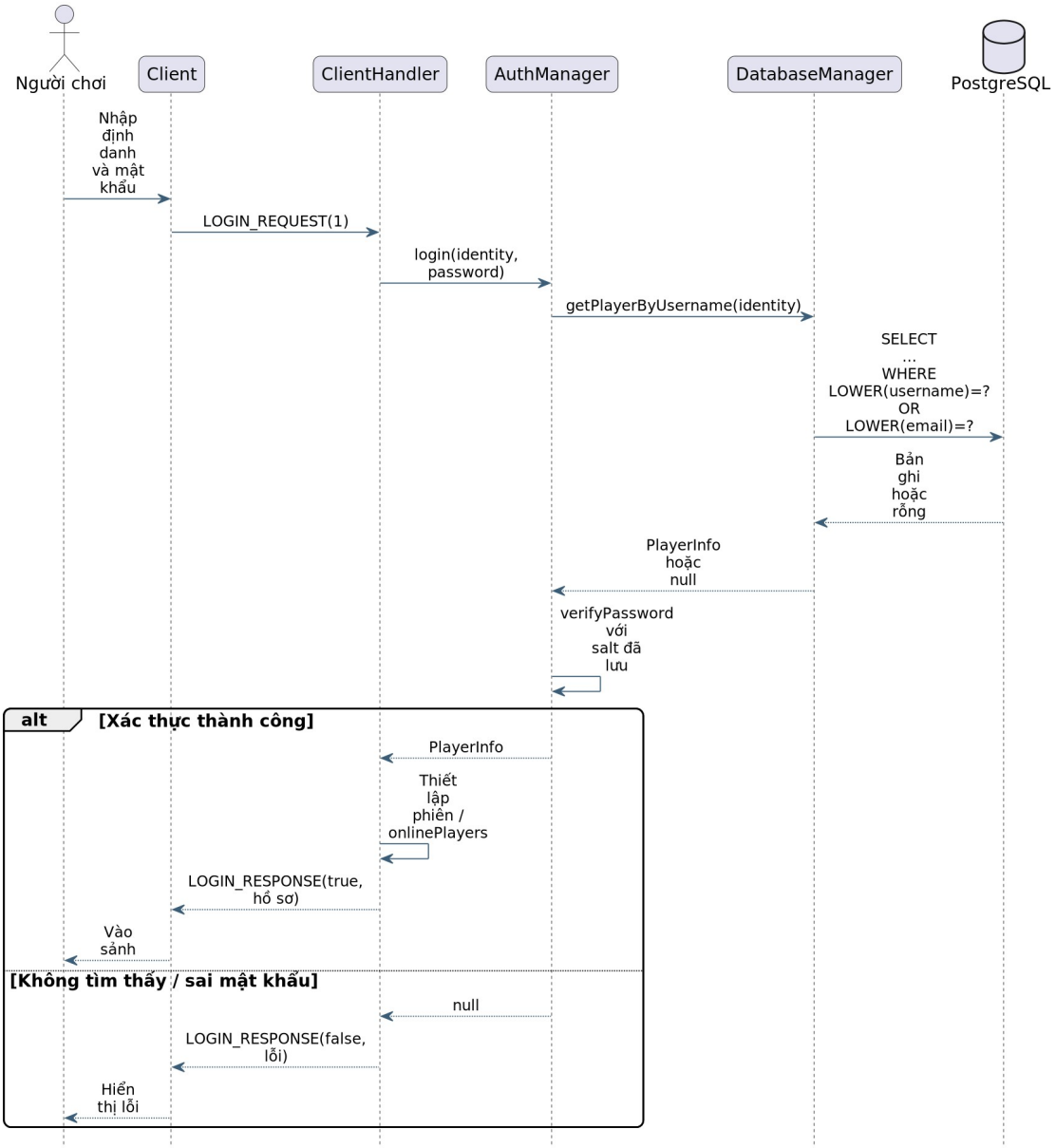
Phạm vi: UC-D04.



Hình 2.3 — Ghi kết quả, EXP và lịch sử. Loại: Activity; dựng từ mã nguồn 00bf1cc.

Biểu đồ 2.4 — Sequence

Phạm vi: UC-D02.



Hình 2.4 — Đăng nhập bằng username hoặc email. Loại: Sequence; dựng từ mã nguồn 00bf1cc.

2.5. Liên hệ triển khai và giới hạn

Phân hệ xác thực và dữ liệu được mô tả bằng Use Case tổng thể, Activity đăng ký, Activity ghi kết quả và Sequence đăng nhập. Các sơ đồ thể hiện cách yêu cầu được kiểm tra, truy vấn và cập nhật; cấu trúc lưu trữ được tóm tắt trong bảng dưới đây.

Bảng	Trường/ràng buộc chính	Use case sử dụng
players	id PK; email/username UNIQUE; password_hash; exp, level, wins, losses; avatar_id, cue_color	UC-D01–UC-D05

Bảng	Trường/ràng buộc chính	Use case sử dụng
match_history	id PK; winner_id và loser_id FK; exp_gained; played_at	UC-D04
friendships	Khóa kép user_id/friend_id; status; created_at	UC-D06
messages	id PK; sender_id/receiver_id FK; content; is_read; sent_at	UC-D07

UNIQUE trên username gốc chưa thay cho UNIQUE trên LOWER(username) khi có yêu cầu đồng thời. Việc lấy username hoặc email trong một truy vấn còn cần kiểm tra trùng chéo định danh. Hai điểm này là nhận xét từ mã/schema; không được đánh dấu như đã có bài thử đồng thời đầy đủ.

PHẦN 3. GAMEPLAY & ENGINE VẬT LÝ

Phạm vi chức năng: 8 use case, 3 biểu đồ.

3.1. Phạm vi và ranh giới trách nhiệm

Phân hệ gameplay và vật lý quản lý bàn chơi, trạng thái bi, hướng ngắm, lực, xoáy, cú đánh, va chạm và xử lý lượt. Ball biểu diễn x/y, vx/vy, spin và các trạng thái active/inPocket/isSinking. PhysicsEngine có sáu lỗ và xử lý 10 bước con trong mỗi update. Các hệ số hiện có gồm ma sát 0,985, phục hồi băng 0,78 và phục hồi bi–bi 0,96.

GameScene gọi update theo AnimationTimer khi physicsRunning. Chưa có bộ tích lũy thời gian cố định 1/60 giây trong phiên bản đang đối chiếu. Máy đánh mô phỏng ngay sau gửi CUE_SHOT; server chỉ chuyển thông số cho đối thủ. Người đánh báo PHYSICS_DONE hoặc CLAIM_WIN theo logic client; server chưa kiểm chứng va chạm bằng một mô phỏng riêng.

MatchSeed dùng xáo 14 bi mục tiêu khác bi 8 và chia hai nhóm bảy bi. Đây là luật phân nhóm của dự án; không thể mô tả là luôn chia nhóm trơn/sọc chuẩn. Cùng seed không chứng minh mọi bi sau cú đánh sẽ khớp ở mọi tốc độ khung hình.

3.2. Danh mục use case

Mã	Tên use case	Tác nhân chính
UC-G01	Khởi tạo bàn và phân nhóm bi	Hai người chơi
UC-G02	Đặt bi cái	Người đang có lượt
UC-G03	Ngắm và kéo lực gậy	Người đang có lượt và người quan sát
UC-G04	Chỉnh xoáy và thực hiện cú đánh	Người đang có lượt
UC-G05	Mô phỏng chuyển động và va chạm	GameScene (tác nhân hệ thống)
UC-G06	Xử lý bi vào lỗ và kết quả lượt	GameScene (tác nhân hệ thống)
UC-G07	Điều phối lượt và hết thời gian	Hai người chơi; GameRoom timer
UC-G08	Luyện tập solo	Người chơi

3.3. Đặc tả use case

UC-G01 — Khởi tạo bàn và phân nhóm bi

Tác nhân: Hai người chơi.

Tiền điều kiện: Client nhận GAME_START với chỉ số người chơi và seed.

Luồng chính:

1. Tạo 16 bi.
2. Xáo 14 bi mục tiêu khác bi 8 theo matchSeed.
3. Chia hai nhóm bảy bi.
4. Thiết lập người có lượt.

Ngoại lệ: READY hiện đếm số packet, nên một người có thể gửi lặp để mở trận; cùng seed chưa bảo đảm mô phỏng luôn đồng nhất.

Hậu điều kiện: Có bàn và nhóm mục tiêu; luật phân nhóm là biến thể của dự án.

Mã nguồn liên quan: GameRoom.playerReady/startGame; GameScene constructors.

UC-G02 — Đặt bi cái

Tác nhân: Người đang có lượt.

Tiền điều kiện: Client cho phép khai cuộc hoặc ball-in-hand.

Luồng chính:

1. Kéo/chọn vị trí.
2. Client giới hạn vùng đặt.
3. Gửi CUE_BALL_PLACED.
4. Server kiểm tra người có lượt rồi chuyển tọa độ.

Ngoại lệ: Không có quyền tại UI thì không đặt; server chưa có snapshot chuẩn để đối chiếu mọi vị trí.

Hậu điều kiện: Client nhận cập nhật vị trí bi cái.

Mã nguồn liên quan: GameScene.repositionCueBall và thao tác chuột; GameRoom.processCueBallPlaced.

UC-G03 — Ngắm và kéo lực gậy

Tác nhân: Người đang có lượt và người quan sát.

Tiền điều kiện: Bi chưa chạy; UI cho phép ngắm.

Luồng chính:

1. Tính góc theo chuột và tỷ lệ kéo.
2. Gửi CUE_AIM có điều tiết.
3. Đối thủ nội suy góc/lực để hiển thị.

Ngoại lệ: Không đúng lượt: server bỏ qua; dữ liệu NaN hiện vẫn được chuyển tiếp nếu gửi trực tiếp.

Hậu điều kiện: Người quan sát thấy trạng thái ngắm; chưa có token/sequence.

Mã nguồn liên quan: GameScene.sendAimUpdateThrottled/startGameLoop; GameRoom.processCueAim.

UC-G04 — Chỉnh xoáy và thực hiện cú đánh

Tác nhân: Người đang có lượt.

Tiền điều kiện: Có hướng/lực hợp lệ ở client và bi cái hoạt động.

Luồng chính:

1. Chỉnh spinX/spinY.
2. Thả chuột.
3. Gửi sáu double CUE_SHOT.
4. Chạy vật lý tại máy đánh.
5. Server chuyển GAME_STATE cho đối thủ.

Ngoại lệ: Cú kéo quá ngắn không tạo cú đánh; server chưa chặn cú đánh trùng trong cùng giai đoạn.

Hậu điều kiện: Hai phía có thông số cú đánh; không có xác nhận server trước mô phỏng máy đánh.

Mã nguồn liên quan: GameScene.onMouseReleased/applyCueShot; NetworkClient.sendCueShot; GameRoom.processCueShot.

UC-G05 — Mô phỏng chuyển động và va chạm

Tác nhân: GameScene (tác nhân hệ thống).

Tiền điều kiện: physicsRunning và danh sách bi đã khởi tạo.

Luồng chính:

1. Mỗi khung hình gọi PhysicsEngine.update.
2. Thực hiện 10 bước con, di chuyển, ma sát, va chạm băng và bi–bi.

Ngoại lệ: Tốc độ khung hình khác nhau có thể thay đổi tiến độ; hiện không có tích lũy bước thời gian cố định.

Hậu điều kiện: Vị trí/vận tốc và hiệu ứng va chạm được cập nhật.

Mã nguồn liên quan: PhysicsEngine.update/resolveWallCollision/resolveBallCollision; Ball.

UC-G06 — Xử lý bi vào lỗ và kết quả lượt

Tác nhân: GameScene (tác nhân hệ thống).

Tiền điều kiện: Bi đang được mô phỏng hoặc vừa dừng.

Luồng chính:

1. Kiểm tra sáu lỗ.
2. Chạy hoạt ảnh chìm.
3. Xác định active.
4. Đánh giá phạm lỗi/giữ lượt và điều kiện bi 8 theo luật triển khai.

Ngoại lệ: Client có thể phát CLAIM_WIN; server chưa kiểm chứng kết luận bằng mô phỏng vật lý riêng.

Hậu điều kiện: Có trạng thái bi và thông tin gửi PHYSICS_DONE hoặc CLAIM_WIN.

Mã nguồn liên quan: PhysicsEngine.checkPockets; Ball.startSinking; GameScene: kết thúc vật lý; GameRoom.endGame.

UC-G07 — Điều phối lượt và hết thời gian

Tác nhân: Hai người chơi; GameRoom timer.

Tiền điều kiện: Trận đã bắt đầu và chưa kết thúc.

Luồng chính:

1. Máy đánh gửi PHYSICS_DONE.
2. Server giữ/đổi lượt.
3. Client đếm 60 giây.
4. Timer server 61,5 giây xử lý chưa đánh.

Ngoại lệ: Timer bị hủy khi có cú đánh; chưa có timeout riêng chờ bản bàn và chưa có ACK hai client.

Hậu điều kiện: Có quyền lượt mới hoặc giữ lượt theo thông tin nhận.

Mã nguồn liên quan: GameRoom.onPhysicsDone/resetTurnTimer; GameScene.startGameLoop.

UC-G08 — Luyện tập solo

Tác nhân: Người chơi.

Tiền điều kiện: Mở chế độ luyện tập từ sảnh.

Luồng chính:

1. Tạo GameScene isPractice.
2. Đặt/ngắm/đánh và mô phỏng cục bộ.
3. Hướng tới dọn bàn.
4. Không tạo đối thủ mạng.

Ngoại lệ: Chưa nghiệm thu mọi trường hợp dọn hết bàn; bố cục đối thủ bị ẩn và unmanaged.

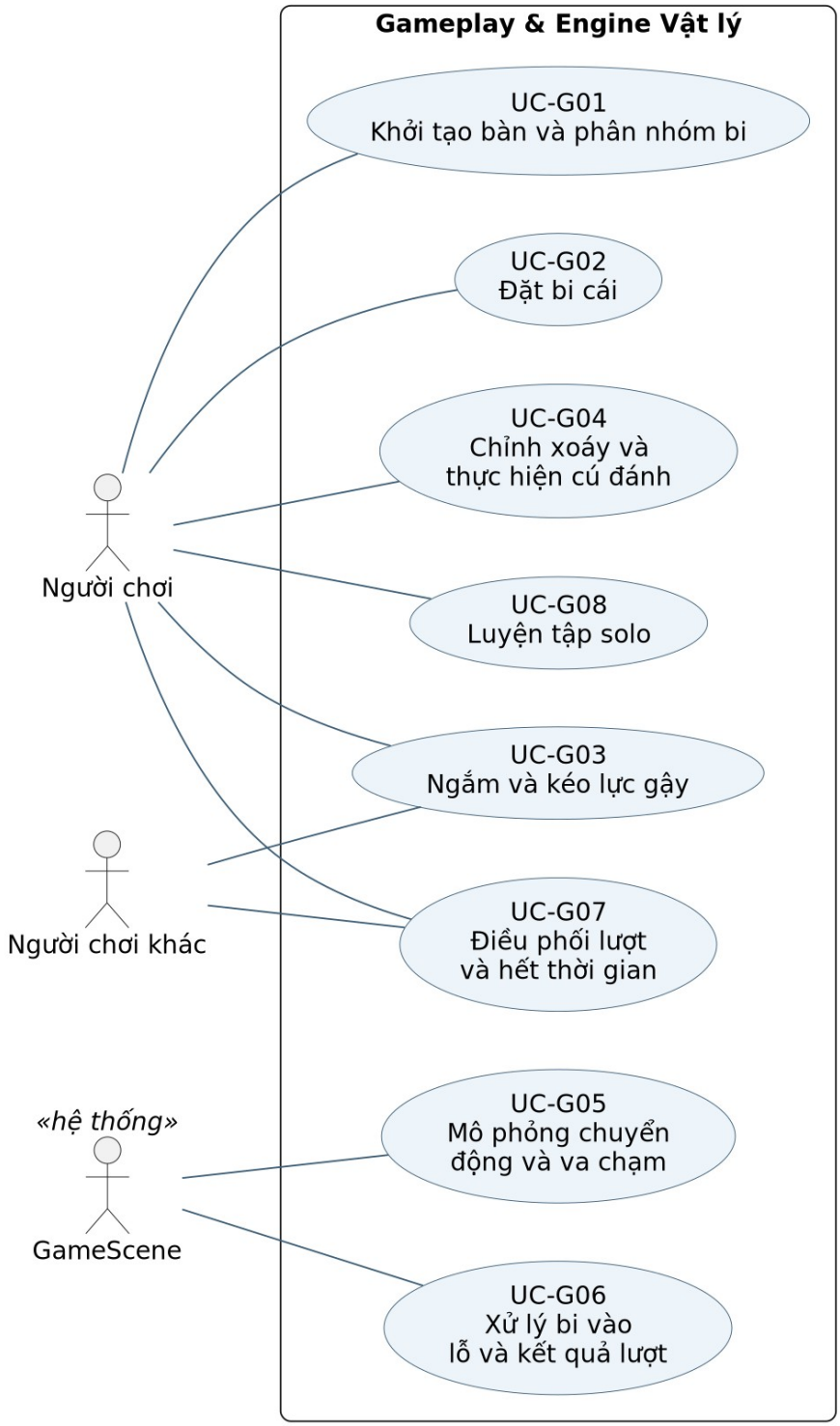
Hậu điều kiện: Người chơi luyện tập trong phiên cục bộ.

Mã nguồn liên quan: SceneManager.showPractice; GameScene practice constructor.

3.4. Biểu đồ UML của phân hệ

Biểu đồ 3.1 — Use Case

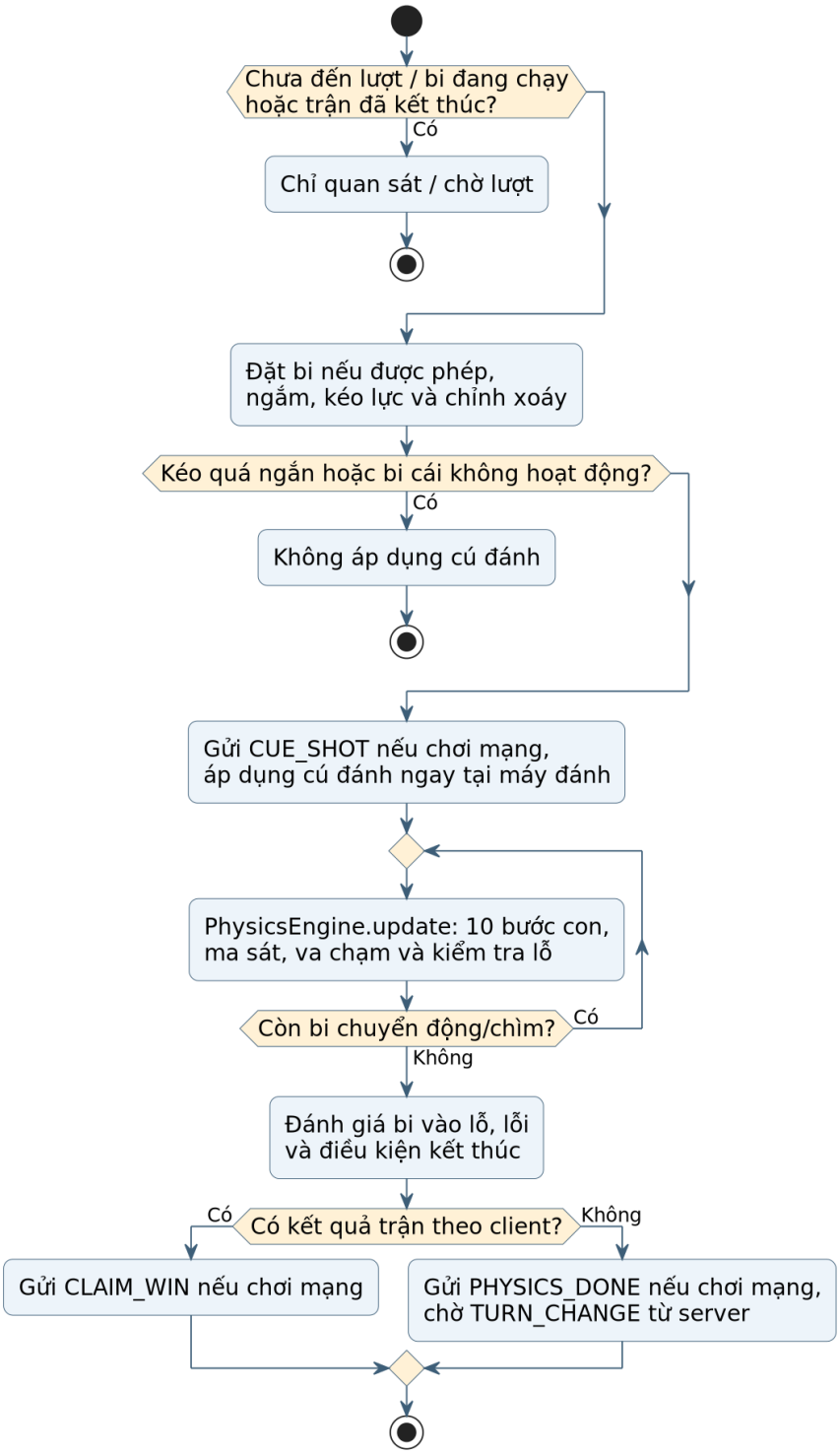
Phạm vi: UC-G01, UC-G02, UC-G03, UC-G04, UC-G05, UC-G06, UC-G07, UC-G08.



Hình 3.1 — Các use case thuộc phân hệ. Loại: Use Case; dựng từ mã nguồn 00bf1cc.

Biểu đồ 3.2 — Activity

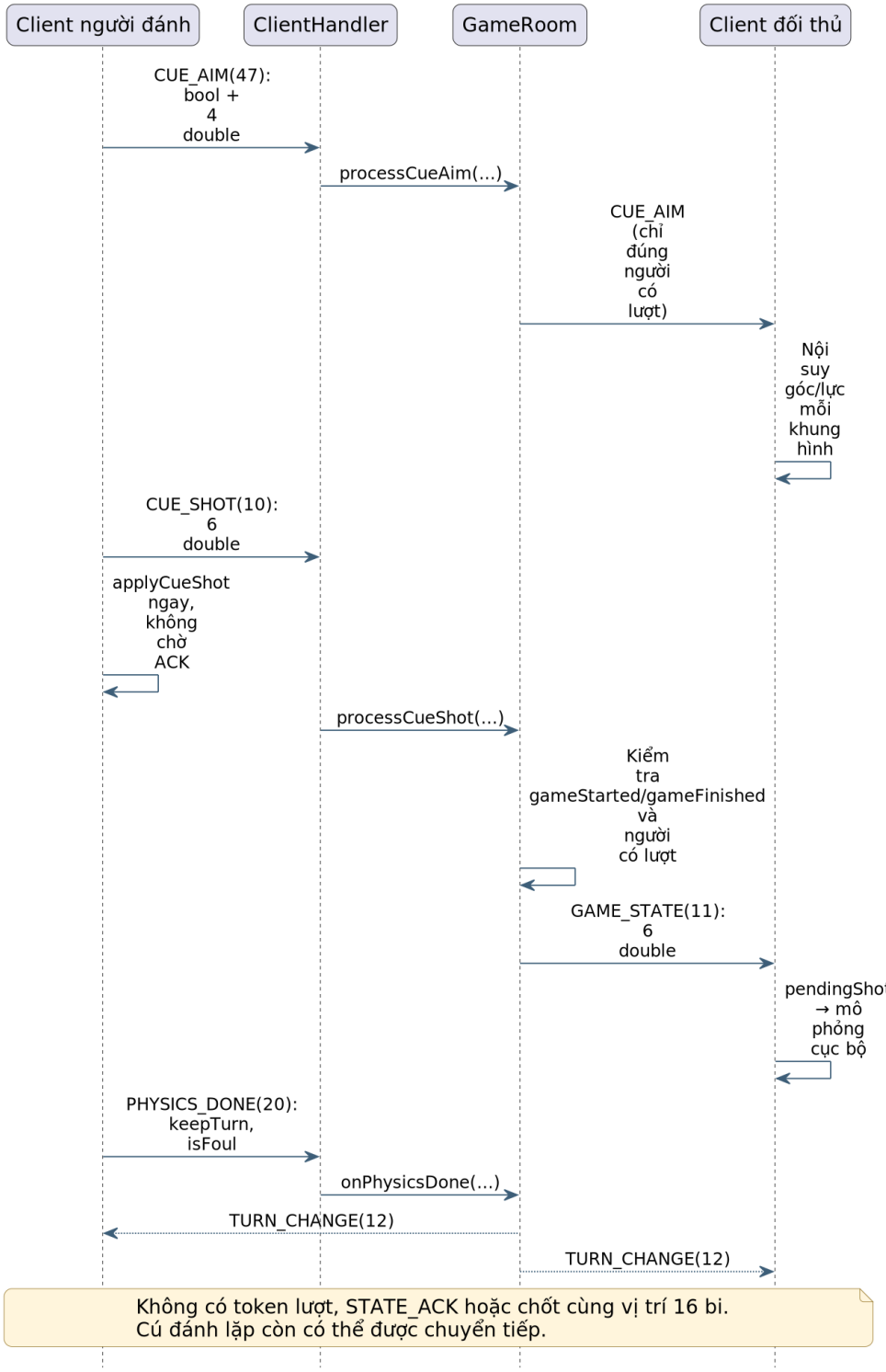
Phạm vi: UC-G02, UC-G03, UC-G04, UC-G05, UC-G06, UC-G07.



Hình 3.2 — Một cú đánh và xử lý cuối lượt. Loại: Activity; dựng từ mã nguồn 00bf1cc.

Biểu đồ 3.3 — Sequence

Phạm vi: UC-G03, UC-G04, UC-G07.



Hình 3.3 — Truyền cú đánh và đổi lượt hiện tại. Loại: Sequence; dựng từ mã nguồn 00bf1cc.

3.5. Liên hệ triển khai và giới hạn

Activity 3.2 mô tả đường đi từ quyền lượt tới mô phỏng và xử lý cuối cú đánh. Sequence 3.3 thể hiện rõ việc người đánh không chờ ACK từ server, server chỉ phát GAME_STATE cho đối thủ và không chốt snapshot cuối lượt.

Giới hạn đã tái hiện ở kiểm tra trước: hai READY của một client có thể đủ mở trận; gói CUE_SHOT lặp được chuyển tiếp; góc NaN có thể được relay. Đó là nội dung cần hoàn thiện, không được lược bỏ để biến sơ đồ thành một thiết kế lý tưởng khác với mã hiện có. Solo UC-G08 là chế độ cục bộ và không cần đối thủ mạng.

PHẦN 4. GIAO DIỆN & CÁ NHÂN HÓA

Phạm vi chức năng: 9 use case, 4 biểu đồ.

4.1. Phạm vi và ranh giới trách nhiệm

Phân hệ giao diện và cá nhân hóa tiếp nhận thao tác, điều hướng màn hình, trình bày trạng thái và áp dụng tùy chọn người chơi. Các lớp tiêu biểu gồm SceneManager, LoginScene, RegisterScene, LobbyScene, PreGameScene, GameScene, ResultScene, LeaderboardScene, SettingsDialog, AvatarManager, CueSkinManager và SoundManager. Giao diện sử dụng phản hồi nghiệp vụ từ server; việc xác thực và lưu dữ liệu vẫn được xử lý tại các thành phần tương ứng.

Lời mời thách đấu dùng popup có owner, modality và alwaysOnTop; thao tác Timeline được đưa về FX Application Thread. Kết quả lời mời dùng toast thay cho Alert chặn màn hình. Solo ẩn vsBadge, oppBox và oppBallsBox, đồng thời loại chúng khỏi bố cục bằng managed=false.

Cá nhân hóa có hai đường xử lý: avatar/skin gửi SETTINGS_UPDATE tới server; âm thanh BGM/SFX/mute áp dụng cục bộ và lưu Preferences. VFX của gậy được điều khiển bằng cờ cục bộ. Không mô tả âm thanh hoặc VFX như packet gameplay mới nếu mã không gửi chúng.

4.2. Danh mục use case

Mã	Tên use case	Tác nhân chính
UC-UI01	Điều hướng màn hình	Người chơi
UC-UI02	Hiển thị và kiểm tra form tài khoản	Người chơi
UC-UI03	Hiển thị sảnh và danh sách bạn bè	Người chơi
UC-UI04	Hiển thị hội thoại chat	Người chơi
UC-UI05	Hiển thị lời mời và màn hình trước trận	Người nhận lời mời hoặc người vừa ghép trận
UC-UI06	Hiển thị bàn chơi và HUD	Người chơi
UC-UI07	Hiển thị kết quả và bảng xếp hạng	Người chơi
UC-UI08	Chọn avatar và skin gậy	Người chơi
UC-UI09	Điều chỉnh âm thanh và hiệu ứng	Người chơi

4.3. Đặc tả use case

UC-UI01 — Điều hướng màn hình

Tác nhân: Người chơi.

Tiền điều kiện: Ứng dụng JavaFX đã khởi tạo.

Luồng chính:

1. SceneManager đổi Scene theo đăng nhập, sảnh, tìm trận, trước trận, game và kết quả.

Ngoại lệ: Callback đến muộn hoặc thay Scene cần quản lý vòng đời; chưa khẳng định mọi luồng chuyển cảnh đã kiểm thử.

Hậu điều kiện: Người dùng ở màn hình phù hợp với thao tác/sự kiện.

Mã nguồn liên quan: ClientMain; SceneManager.

UC-UI02 — Hiển thị và kiểm tra form tài khoản

Tác nhân: Người chơi.

Tiền điều kiện: Đang ở LoginScene hoặc RegisterScene.

Luồng chính:

1. Nhập trường.
2. Kiểm tra đủ dữ liệu, email, tên, mật khẩu xác nhận.
3. Hiển thị lỗi hoặc gọi NetworkClient.

Ngoại lệ: Regex email hiện chặn đuôi miền dài hơn 7 chữ cái; kiểm tra UI không thay kiểm tra server.

Hậu điều kiện: Có thông báo hoặc yêu cầu đăng nhập/đăng ký được gửi.

Mã nguồn liên quan: LoginScene; RegisterScene.doRegister.

UC-UI03 — Hiển thị sảnh và danh sách bạn bè

Tác nhân: Người chơi.

Tiền điều kiện: Đã đăng nhập và mở LobbyScene.

Luồng chính:

1. Hiển thị hồ sơ.
2. Refresh bạn bè 3,5 giây.
3. Không dựng lại dữ liệu giống.
4. Mở menu Nhắn tin/Thách đấu/Xóa bạn.

Ngoại lệ: Đang mở menu: hoãn cập nhật; chuỗi A → B → A còn có thể áp dụng B cũ khi đóng.

Hậu điều kiện: Sảnh/menu cập nhật theo callback; hạn chế hiện tại được ghi nhận.

Mã nguồn liên quan: LobbyScene.updateFriendList/createFriendItem/startFriendAutoRefresh.

UC-UI04 — Hiển thị hội thoại chat

Tác nhân: Người chơi.

Tiền điều kiện: Chọn một người để xem chat.

Luồng chính:

1. Mở panel, yêu cầu lịch sử.
2. Hiển thị tin tự gửi và nhận.
3. Cuộn xuống.
4. Đối chiếu dòng khi lịch sử tới.

Ngoại lệ: Hai tin giống nhau cùng phút có thể bị gộp; lịch sử trả muộn có thể vào sai cuộc trò chuyện.

Hậu điều kiện: Có hội thoại hiển thị, chưa bảo đảm nhận diện mọi tin bằng ID.

Mã nguồn liên quan: LobbyScene.showChatView/appendChatMessage/setupNetworkCallback.

UC-UI05 — Hiển thị lời mời và màn hình trước trận

Tác nhân: Người nhận lời mời hoặc người vừa ghép trận.

Tiền điều kiện: Có CHALLENGE_RECEIVE hoặc MATCH_FOUND.

Luồng chính:

1. Hiện popup nổi và đếm 10 giây.
2. Gửi lựa chọn.
3. Dùng toast cho kết quả.
4. Khi MATCH_FOUND thì dừng refresh và chuyển PreGameScene.

Ngoại lệ: Đếm hết tự từ chối; callback UI được chuyển về FX thread; server còn thời hạn riêng 15 giây.

Hậu điều kiện: Popup đóng hoặc chuyển màn hình trước trận.

Mã nguồn liên quan: LobbyScene.showChallengeReceivedDialog/showToast; PreGameScene.

UC-UI06 — Hiển thị bàn chơi và HUD

Tác nhân: Người chơi.

Tiền điều kiện: Có GameScene đối kháng hoặc luyện tập.

Luồng chính:

1. Vẽ bàn/bi/gậy, lượt, lực, xoáy, thời gian và ping/FPS.
2. Solo ẩn thông tin đối thủ và bỏ khung trống.

Ngoại lệ: Số FPS/ping trên dữ liệu giả lập không phải benchmark; không tự quyết định luật từ thông tin hiển thị.

Hậu điều kiện: Khung hình thể hiện trạng thái hiện có của client.

Mã nguồn liên quan: GameScene.build/startGameLoop; Canvas; AvatarManager.

UC-UI07 — Hiển thị kết quả và bảng xếp hạng

Tác nhân: Người chơi.

Tiền điều kiện: Có phản hồi GAME_OVER hoặc dữ liệu leaderboard.

Luồng chính:

1. Dựng kết quả thắng/thua/EXP.
2. Hiển thị các dòng xếp hạng do server trả.
3. Cho quay về màn hình tiếp theo.

Ngoại lệ: Không có dữ liệu hoặc lỗi mạng cần thông báo; chưa có nghiệm thu toàn bộ UI bảng xếp hạng trong lần kiểm tra trước.

Hậu điều kiện: Người dùng xem kết quả/xếp hạng từ dữ liệu server.

Mã nguồn liên quan: ResultScene; LeaderboardScene; SceneManager.

UC-UI08 — Chọn avatar và skin gậy

Tác nhân: Người chơi.

Tiền điều kiện: Mở SettingsDialog và có hồ sơ hiện tại.

Luồng chính:

1. Chọn avatar/skin.
2. Xem preview và yêu cầu cấp độ.
3. Lưu bằng SETTINGS_UPDATE.
4. Gửi yêu cầu để server kiểm tra quyền và lưu dữ liệu.

Ngoại lệ: Skin khóa bị hạn chế; server kiểm tra lại cấp; lỗi parser SETTINGS_RESPONSE có thể cản xác nhận thành công.

Hậu điều kiện: Lựa chọn được gửi; trạng thái lưu thành công cần phản hồi đúng.

Mã nguồn liên quan: SettingsDialog; AvatarManager; CueSkinManager; NetworkClient.sendSettingsUpdate.

UC-UI09 — Điều chỉnh âm thanh và hiệu ứng

Tác nhân: Người chơi.

Tiền điều kiện: Đang mở tab âm thanh/cài đặt.

Luồng chính:

1. Đổi BGM/SFX/mute.
2. Thử âm thanh.
3. Bật/tắt VFX gậy.
4. SoundManager áp dụng và lưu các tùy chọn âm thanh vào Preferences.

Ngoại lệ: Thiếu tài nguyên hoặc thiết bị âm thanh ảnh hưởng kết quả; VFX là cờ cục bộ, không khẳng định được lưu như âm thanh.

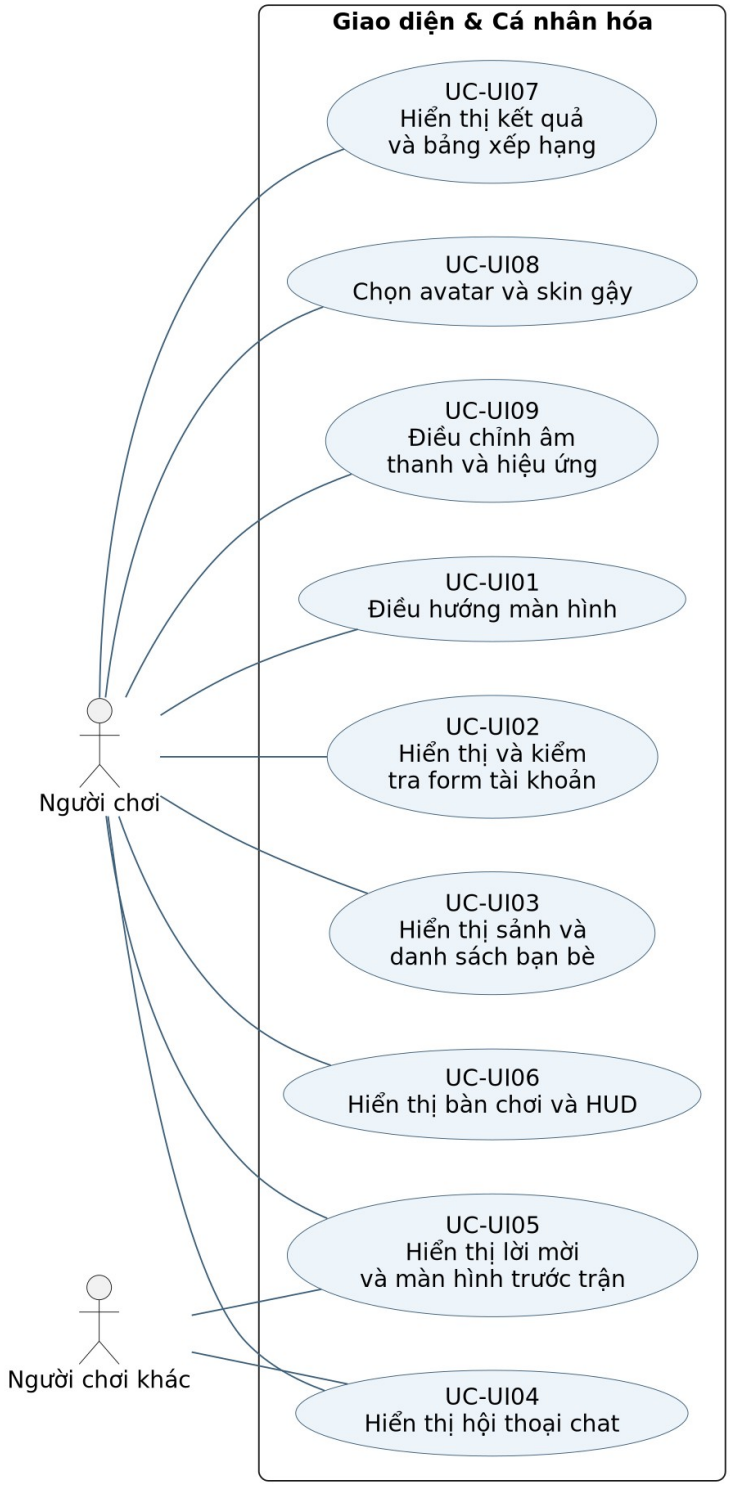
Hậu điều kiện: Âm thanh/hiệu ứng thay đổi cục bộ, không tạo packet gameplay.

Mã nguồn liên quan: SettingsDialog.buildAudioTab; SoundManager; CueSkinManager.setVfxEnabled.

4.4. Biểu đồ UML của phân hệ

Biểu đồ 4.1 — Use Case

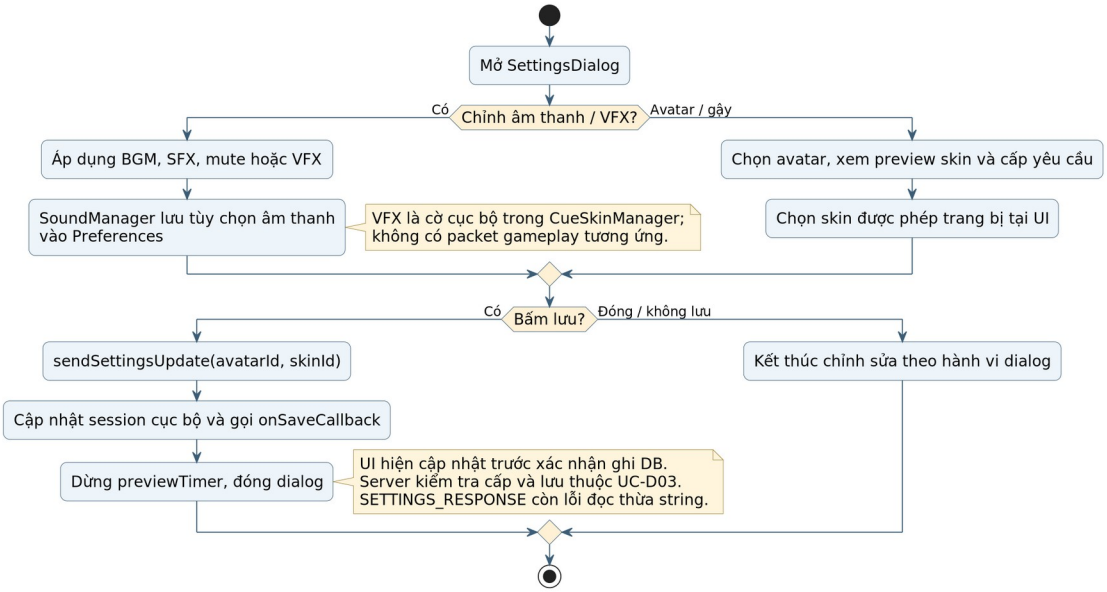
Phạm vi: UC-UI01, UC-UI02, UC-UI03, UC-UI04, UC-UI05, UC-UI06, UC-UI07, UC-UI08, UC-UI09.



Hình 4.1 — Các use case thuộc phân hệ. Loại: Use Case; dựng từ mã nguồn 00bf1cc.

Biểu đồ 4.2 — Activity

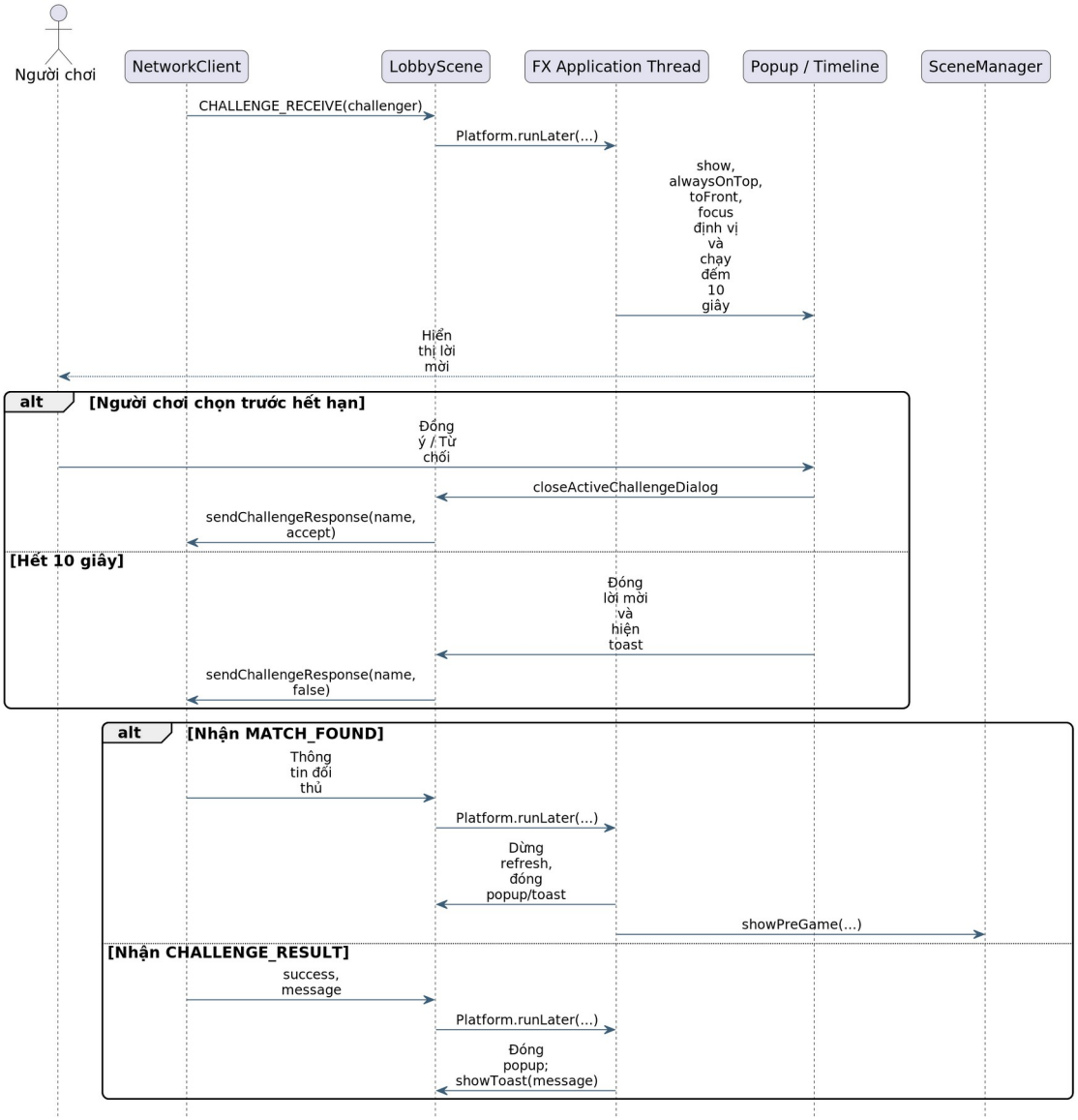
Phạm vi: UC-UI08, UC-UI09.



Hình 4.2 — Cá nhân hóa avatar, gậy và âm thanh. Loại: Activity; dựng từ mã nguồn 00bf1cc.

Biểu đồ 4.3 — Sequence

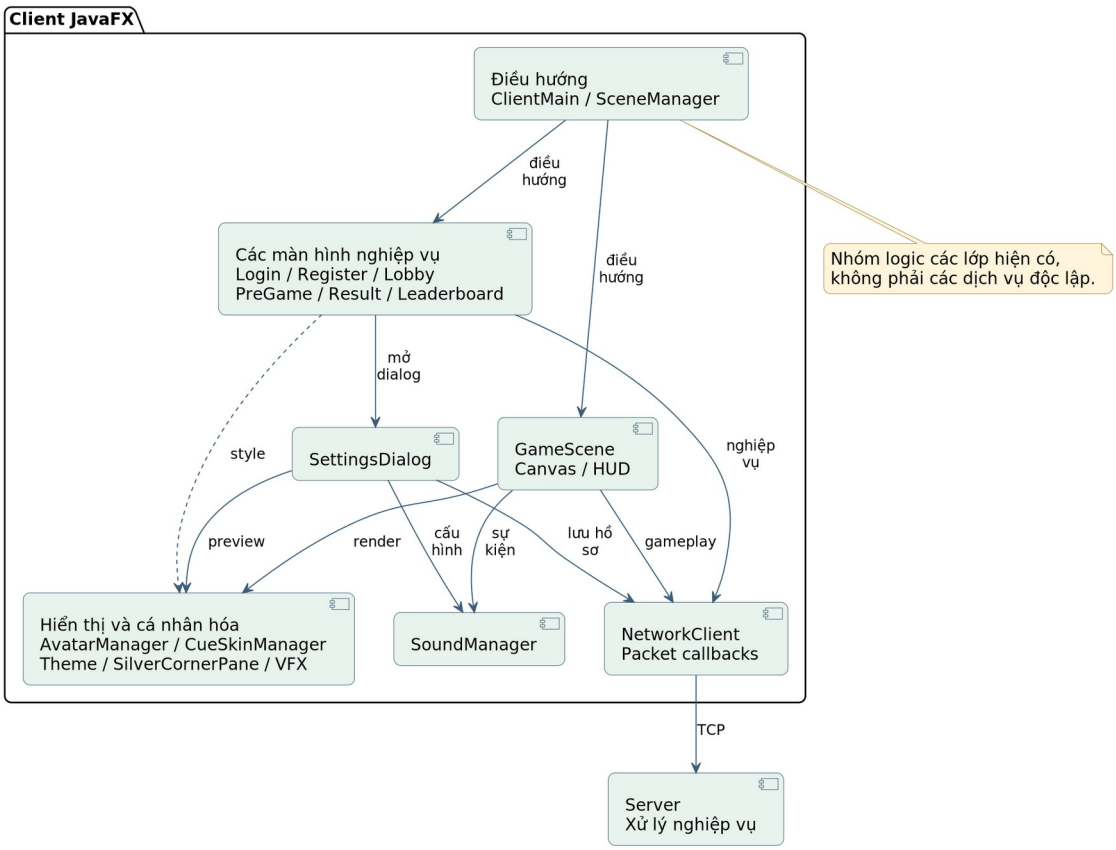
Phạm vi: UC-UI01, UC-UI03, UC-UI05.



Hình 4.3 — Popup thách đấu, toast và chuyển cảnh. Loại: Sequence; dựng từ mã nguồn 00bf1cc.

Biểu đồ 4.4 — Component

Phạm vi: UC-UI01, UC-UI02, UC-UI03, UC-UI04, UC-UI05, UC-UI06, UC-UI07, UC-UI08, UC-UI09.



Hình 4.4 — Thành phần giao diện và cá nhân hóa. Loại: Component; dựng từ mã nguồn 00bf1cc.

4.5. Liên hệ triển khai và giới hạn

Component 4.4 mô tả các nhóm lớp và phụ thuộc trong ứng dụng; các hộp không đại diện cho microservice hoặc tiến trình triển khai riêng. NetworkClient kết nối giao diện với server. Các yêu cầu lưu hồ sơ được xử lý ở phía server, trong khi GameScene kết hợp HUD với engine vật lý để thể hiện trạng thái bàn chơi.

Activity 4.2 phản ánh saveSettings hiện cập nhật session/callback cục bộ rồi đóng dialog, không đợi xác nhận DB. Phản hồi SETTINGS_RESPONSE hiện có lỗi đọc thừa string ở client. Vì vậy hậu điều kiện “đã hiển thị lựa chọn” không đồng nghĩa “đã lưu thành công và được xác nhận”. Sequence 4.3 tập trung popup, phản hồi, toast và chuyển cảnh trước trận.

TRIỂN KHAI VÀ HƯỚNG DẪN SỬ DỤNG

Môi trường và cấu trúc bàn giao

Hạng mục	Cấu hình được mã/script sử dụng	Ghi chú triển khai
Java	Các script Windows trỏ JDK 17	Máy build cần javac, jar và jpackage khi đóng gói EXE
JavaFX	SDK Windows x64 17.0.13 theo đường dẫn script	Đặt tại client/lib/javafx-sdk-17.0.13; có cả JAR và DLL
PostgreSQL	Database billiards_db, JDBC ở localhost:5432 theo mã mặc định	Tạo schema trước khi dùng tài khoản và dữ liệu
JDBC driver	server/lib/postgresql-42.7.3.jar	Phải có trong classpath của server
Mạng ứng dụng	TCP 12345; UDP discovery 12346	Hai client phải truy cập được địa chỉ server
Đóng gói một EXE	jpackage, tar.exe và csc.exe theo script	Tạo trên Windows; chưa được nghiệm thu trong môi trường báo cáo

Thư mục client chứa giao diện, engine, protocol, assets và script build; server chứa phần kết nối, nghiệp vụ, dữ liệu và protocol; database chứa schema; reports chứa tài liệu và minh chứng. Các thư mục out/dist, SDK lớn và file EXE bị loại khỏi Git theo .gitignore hiện tại. Vì vậy, tải mã nguồn ZIP từ GitHub không đồng nghĩa đã tải được một bản EXE mới.

Chuẩn bị database và server

Với một môi trường phát triển mới, cài PostgreSQL và chạy schema bằng psql từ thư mục gốc dự án:

```
psql -U postgres -f database/schema.sql
```

Script có CREATE DATABASE và lệnh riêng của psql là \c. Nếu database đã tồn tại, cần chọn đúng database và thực hiện phần tạo bảng phù hợp; không xóa dữ liệu đang dùng để lặp lại bước khởi tạo. Nếu dùng pgAdmin, tạo/chọn database trước và chạy phần SQL tạo bảng, không dán nguyên lệnh \c vào Query Tool.

Kiểm tra DB_URL, DB_USER và DB_PASSWORD trong DatabaseManager.java để khớp cấu hình cục bộ, sau đó biên dịch lại server nếu đã đổi giá trị. Phiên bản này đặt cấu hình trong mã; báo cáo không giả định có hỗ trợ biến môi trường hoặc tệp cấu hình ngoài. Cần có thông báo kết nối PostgreSQL thành công; việc tiến trình server vẫn chạy không có nghĩa đăng ký hay lưu dữ liệu sẽ hoạt động khi DB lỗi.

Mở Command Prompt tại thư mục server, chạy compile.bat, sau đó run_server.bat. Các script này dùng thư mục làm việc hiện tại và trỏ JDK tại C:\Program Files\Java\jdk-17. Người triển khai cần điều chỉnh đường dẫn nếu JDK nằm ở nơi khác. Giữ cửa sổ server để theo dõi lỗi trong quá trình thử nghiệm.

Biên dịch và đóng gói client EXE

Máy phát triển cần đặt JavaFX Windows SDK đúng thư mục mà script yêu cầu. Từ thư mục client, có thể chạy compile.bat rồi run_client.bat để chẩn đoán trước khi đóng gói. Script chạy ưu tiên BilliardsClient.jar nếu file này tồn tại, nên phải build lại JAR khi cần kiểm tra mã mới; chỉ thay file .java không cập nhật binary đã có.

Đối với người dùng cuối ưu tiên EXE, quy trình đóng gói có sẵn trong mã là:

1. Chạy client\build_single_exe.bat trên Windows có các công cụ yêu cầu.
2. Script gọi make_exe.bat, biên dịch client, dùng jpackage tạo app-image có runtime, sao chép JavaFX DLL và assets rồi tạo ZIP.
3. Trình biên dịch C# nhúng ZIP vào launcher, tạo client\dist\8BallPool_Game.exe và sao chép ra thư mục gốc.
4. Chuyển EXE được tạo thành công sang máy kiểm tra sạch, chạy và xác minh kết nối, âm thanh, tài nguyên và phiên bản nội dung.

Launcher giải nén dữ liệu vào %LOCALAPPDATA%\BilliardsClubGame rồi khởi chạy BilliardsClient.exe trong đó. Đường app-image cũng có thể dùng trực tiếp, nhưng cần giữ nguyên thư mục runtime, app và assets cùng launcher. Các bước trên là hướng dẫn từ script hiện có; lần viết báo cáo này không tạo hay phát hành EXE và chưa xác minh gói trên Windows sạch.

Quy trình sử dụng chung của hai người chơi

Hai máy mở cùng phiên bản client và chọn server. Khi thử hai client trên cùng một máy, có thể dùng localhost; khi thử hai máy khác nhau, client phải nhập IP LAN của máy chạy server. localhost ở máy thứ hai sẽ trỏ về chính máy đó. Nếu khám phá tự động không thấy server, kiểm tra IP, kết nối mạng và quyền truy cập cổng trước khi kết luận lỗi giao diện.

Người chơi đăng ký tài khoản, đăng nhập bằng tên hoặc email rồi vào sảnh. Tại sảnh có thể tìm bạn, gửi lời mời kết bạn, chat, tìm trận hoặc thách đấu trực tiếp. Khi người nhận chấp nhận và server tạo phòng, hai bên chuyển sang phần trước trận và thao tác sẵn sàng. Chỉ một người có lượt đánh; người còn lại quan sát.

Trên bàn chơi, người có lượt ngắm gậy, kéo lực và thả để đánh khi điều kiện cho phép. Bộ chọn điểm chạm trên bi cái điều chỉnh xoáy; chuột phải hoặc nhấp đúp trên vùng chọn có đường xử lý đặt lại tâm. Khi có quyền đặt bi cái, chọn vị trí theo giới hạn bàn. Sau cú đánh, chờ bi dừng và phản hồi chuyển lượt. Luyện tập solo dùng mô phỏng cục bộ, không cần đối thủ.

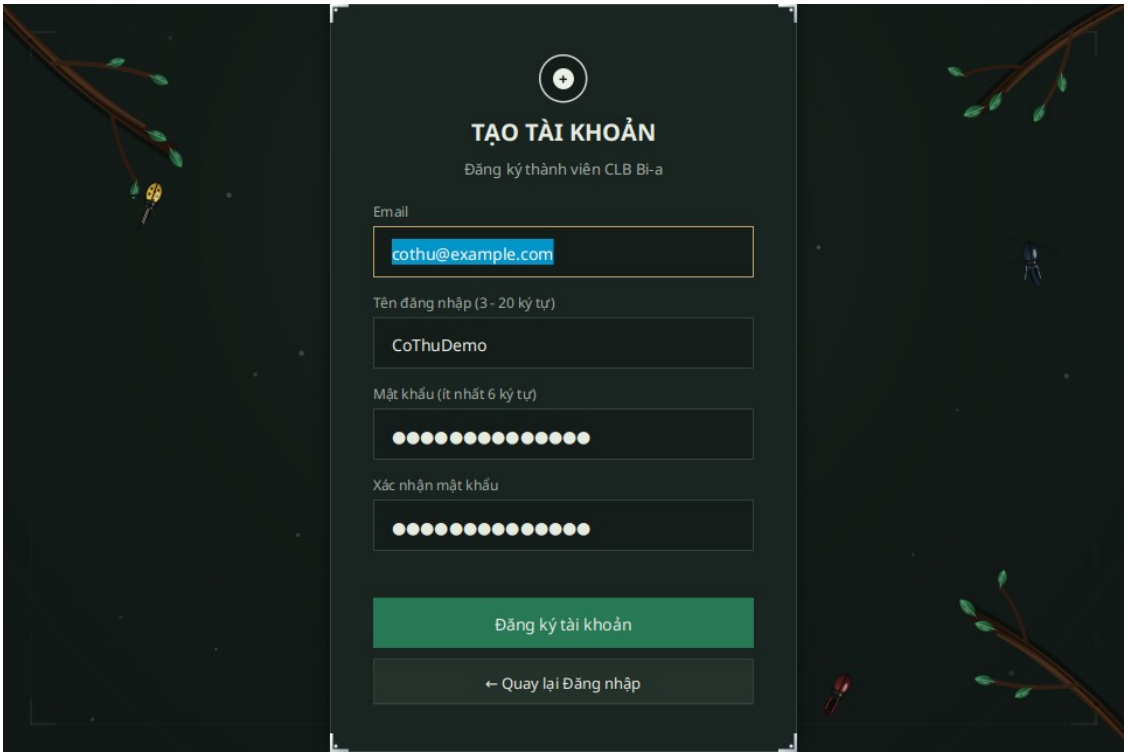
Avatar và skin được chỉnh qua SettingsDialog; skin còn phụ thuộc level được server kiểm tra. BGM, SFX và mute là tùy chọn cục bộ. Do lỗi parser SETTINGS_RESPONSE còn tồn tại, thao tác hiển thị thay đổi ngay tại client chưa đủ để kết luận hồ sơ đã được lưu và phản hồi hoàn tất; cần xác minh lại ở DB hoặc phiên đăng nhập sau trong đợt nghiệm thu.

Minh họa giao diện sản phẩm

Bảy hình dưới đây là ảnh JavaFX thực từ mã 00bf1cc, chụp ngày 26/10/2026 trên Linux/Xvfb. Tài khoản và nội dung là dữ liệu mẫu, transport dùng fixture để điều khiển callback. Chúng minh họa bố cục và phản hồi giao diện; không phải ảnh hai máy Windows thi đấu đầu cuối. Chỉ số ping/FPS trong ảnh không được dùng làm kết quả benchmark. Các ảnh được đánh số M1–M7 riêng với 14 biểu đồ UML.

M1. Form đăng ký

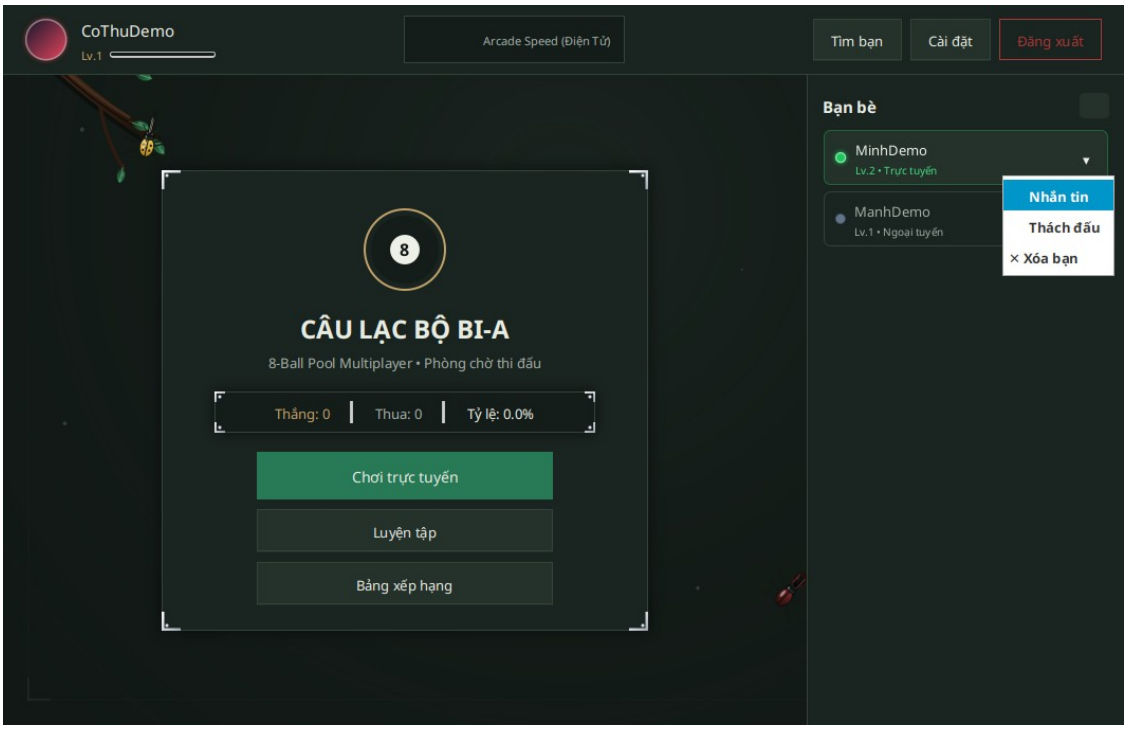
Form nhận thông tin tài khoản và phản hồi kiểm tra đầu vào, liên hệ UC-D01 và UC-UI02. Kiểm tra client hỗ trợ trải nghiệm nhập liệu nhưng không thay thế quy tắc xác thực đầu vào ở server.



Hình M1 — Form đăng ký JavaFX; dữ liệu mẫu và transport giả lập, ngày 26/10/2026.

M2. Sảnh và menu bạn bè

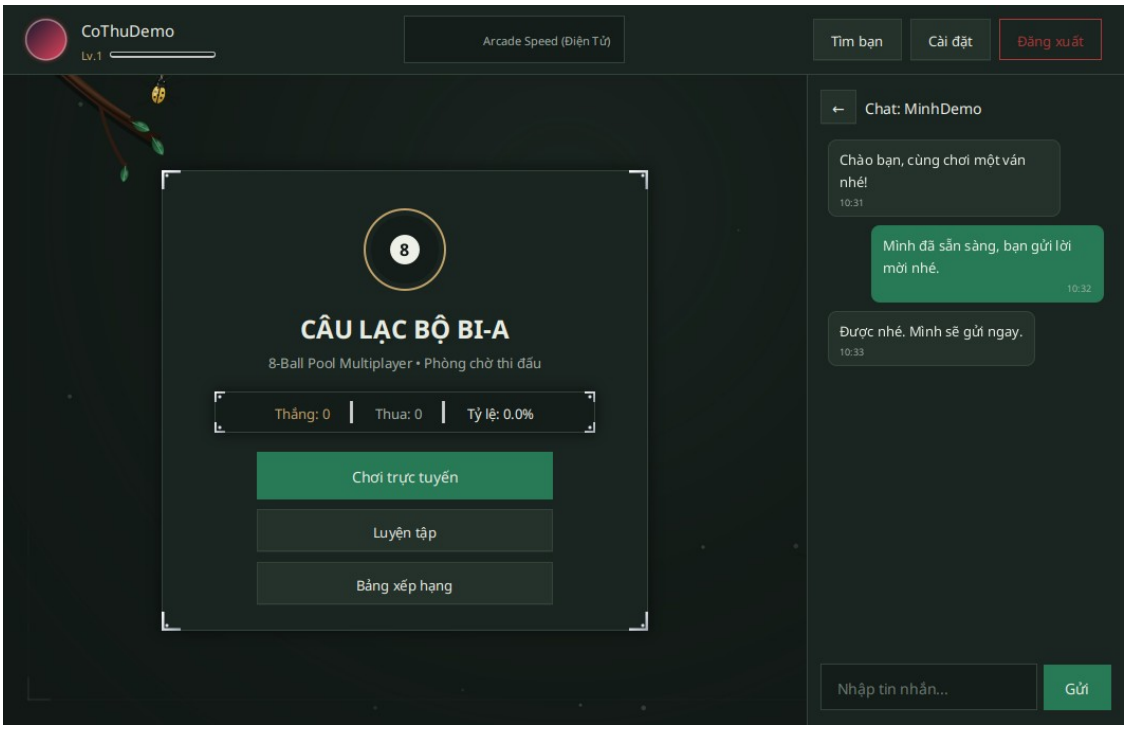
Menu bạn bè cung cấp ngữ cảnh thao tác trong sảnh, liên hệ UC-D06 và UC-UI03. Kịch bản kiểm tra có xác nhận menu được giữ khi refresh và cũng ghi nhận tình huống áp dụng dữ liệu chờ đã cũ.



Hình M2 — Menu bạn bè trên giao diện thật; fixture dữ liệu, ngày 26/10/2026.

M3. Hội thoại chat

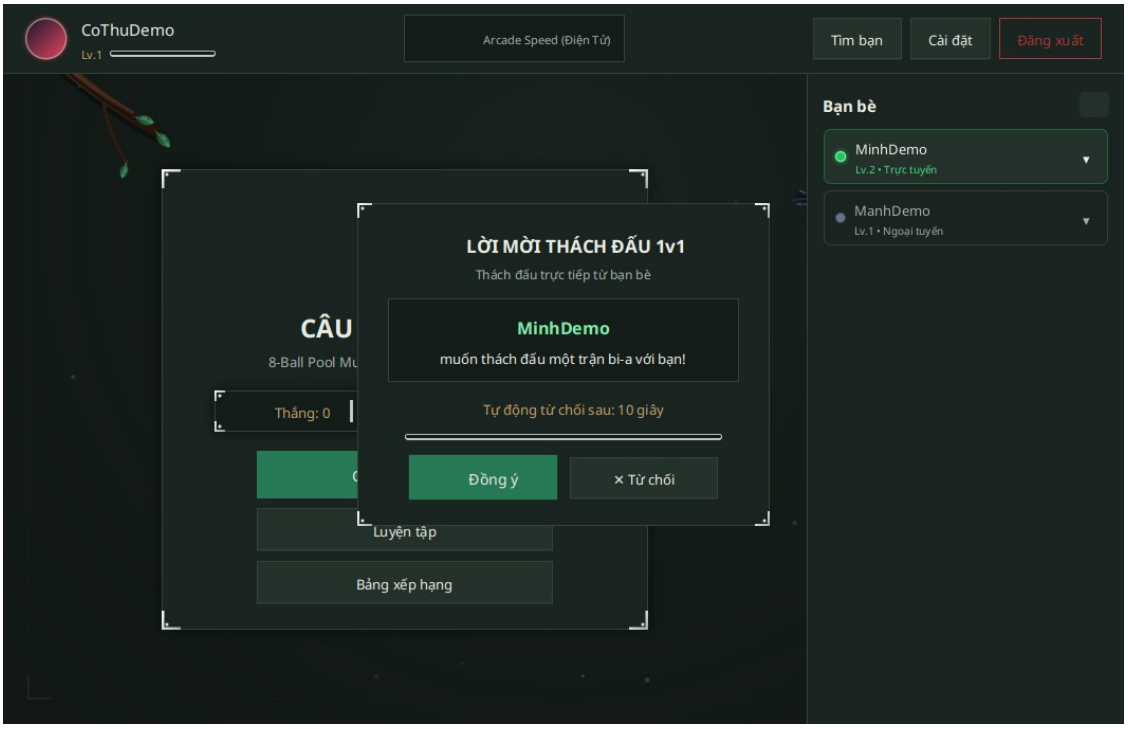
Giao diện hội thoại biểu diễn lịch sử và tin nhắn, liên hệ UC-D07 và UC-UI04. Ảnh thể hiện cách trình bày, không chứng minh việc nhận lịch sử bất đồng bộ đã đúng cho mọi lần chuyển bạn chat.



Hình M3 — Hội thoại với nội dung mẫu trên JavaFX, ngày 26/10/2026.

M4. Lời mời thách đấu

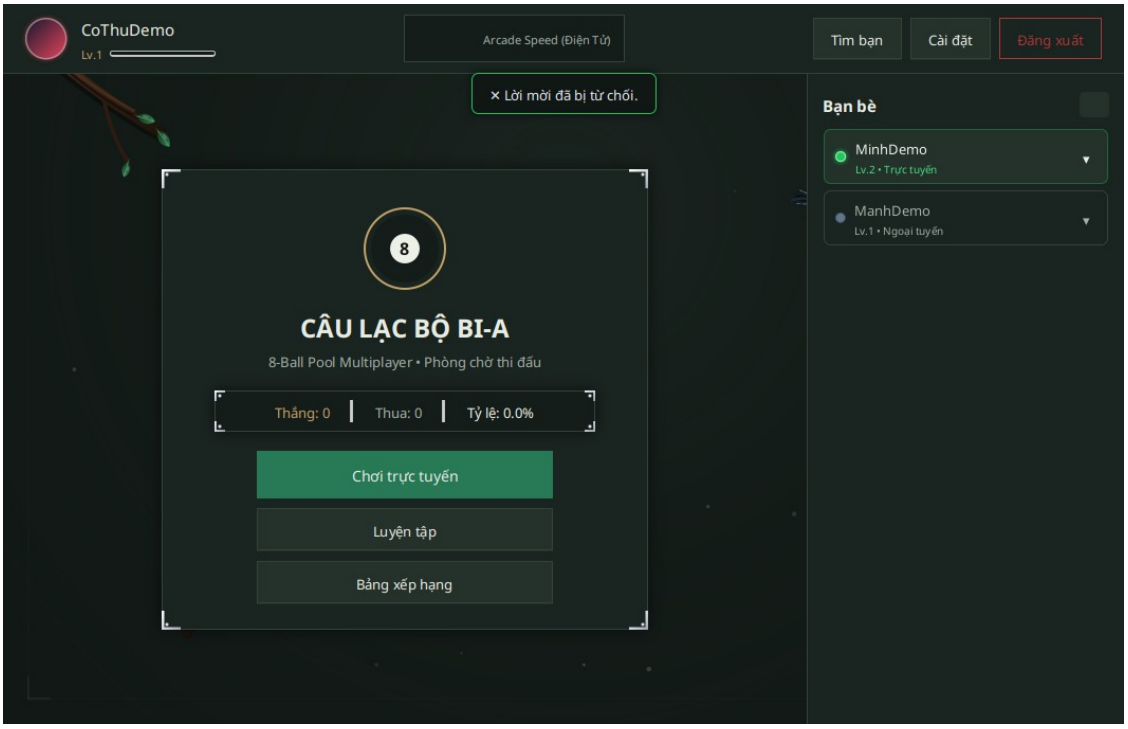
Popup lời mời nằm trên cửa sổ chủ và có bộ đếm; liên hệ UC-C06 và UC-UI05. Server và client có mốc timeout khác nhau, lần lượt 15 và 10 giây trong mã đối chiếu.



Hình M4 — Popup thách đấu trên cửa sổ chủ; callback giả lập, ngày 26/10/2026.

M5. Phản hồi bằng toast

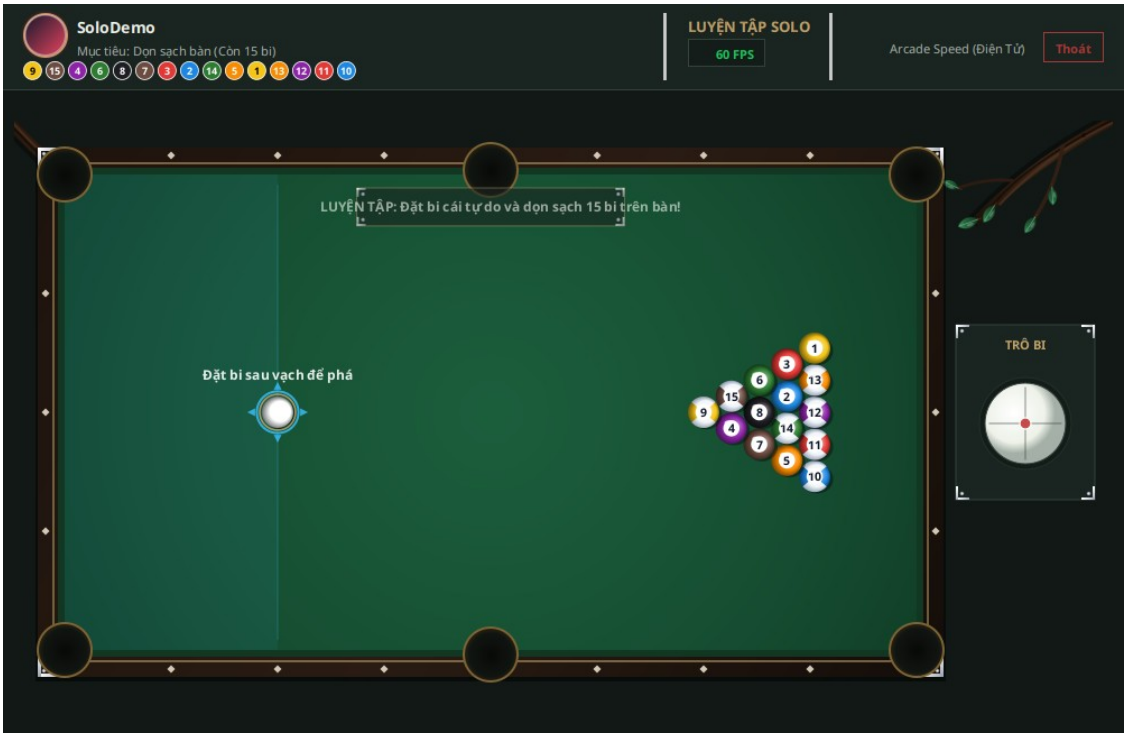
Kết quả lời mời dùng thông báo toast, giúp hiển thị phản hồi mà không mở thêm Alert chặn giao diện. Đây là minh họa cho nhánh phản hồi của UC-UI05.



Hình M5 — Toast kết quả lời mời trên JavaFX, ngày 26/10/2026.

M6. Chế độ luyện tập

Solo ẩn và loại khung đối thủ khỏi bố cục, liên hệ UC-G08 và UC-UI06. Ảnh minh họa việc loại khoảng trống; chưa thay cho nghiệm thu toàn bộ quy tắc vật lý và kết thúc bài luyện tập.



Hình M6 — Bàn luyện tập solo, không có khung đối thủ trống, ngày 26/10/2026.

M7. Bàn chơi và hiển thị hướng gậy

GameScene tiếp nhận mẫu CUE_AIM từ callback có kiểm soát và hiển thị trạng thái tương ứng, liên hệ UC-G03 và UC-UI06. Để nghiệm thu ngắm liên tục, vẫn cần hai client thật với mạng có độ trễ và thay đổi lượt, không chỉ callback đơn lẻ.



Hình M7 — GameScene nhận mẫu ngắm qua fixture; không phải phép đo đồng bộ hai máy, ngày 26/10/2026.

Các sự cố triển khai thường gặp

Hiện tượng	Nội dung cần kiểm tra	Kết quả cần có trước khi tiếp tục
Windows hỏi ứng dụng để mở JAR	Người dùng đang mở file .jar, chưa chạy bản EXE đã đóng gói	Dùng EXE từ bước build thành công hoặc script Java để chẩn đoán
Tải GitHub nhưng không thấy EXE mới	.gitignore loại *.exe và dist; source ZIP không phải gói phát hành	Có gói phát hành riêng hoặc build trên Windows
Không tìm thấy JavaFX/JVM	SDK, kiến trúc x64, runtime và DLL đi kèm	App mở trên máy kiểm tra với đủ tài nguyên
Không kết nối được server	Server đang chạy, IP đúng, TCP 12345 truy cập được	Có kết nối trước khi thử đăng nhập
Đăng ký/đăng nhập lỗi	Database, JDBC classpath, cấu hình và bảng dữ liệu	Server kết nối DB và có phản hồi xác thực
Mã mới nhưng hành vi cũ	JAR/EXE chưa được build lại hoặc đang chạy gói khác	Ghi nhận phiên bản binary đang thử

KIỂM THỬ VÀ ĐÁNH GIÁ CHUNG

Phương pháp và môi trường kiểm chứng

Bộ minh chứng được lưu tại reports/evidence/00bf1cc, thực thi ngày 26/10/2026 trên môi trường Linux. Biên dịch dùng JDK 21 với –release 17, gồm 9 file Java server và 26 file Java client; cả hai thành công. Kiểm tra giao thức dùng server thực và PostgreSQL phát triển cục bộ. Kiểm tra giao diện dùng các lớp JavaFX thật, Xvfb và transport giả lập.

Báo cáo tổng ngày 28/10 kế thừa các log này theo đúng phiên bản và phạm vi; không công bố một lần kiểm thử ứng dụng mới. Kiểm tra tài liệu gồm tính đầy đủ của 31 mã UC, phân bố 14 UML, nguồn ảnh, các file xuất và bố cục. Mã thoát 0 của script mô tả hiện trạng chỉ có nghĩa các assertion đã hoàn tất; một assertion có thể cố ý xác nhận việc tái hiện lỗi.

Lần chạy giao thức đầu tiên dừng ở tình huống phản hồi sai tên vì chưa bảo đảm timer đã được tạo. Lần sau thêm khoảng chờ 250 ms và tái hiện trạng thái bận còn lưu. Cả log lần đầu và lần hoàn tất đều được giữ; không gộp chúng thành một lần chạy hoàn hảo.

Kết quả thực thi đã có

Mã	Trường hợp và nhóm UC	Quan sát thực tế	Đánh giá
KT01	Biên dịch client/server	9 file server, 26 file client biên dịch thành công	Đạt bước build trong môi trường đã nêu
KT02	Đăng ký và đăng nhập, D01–D02	Tài khoản tạm đăng ký và đăng nhập tên khác hoa/thường	Đạt luồng đã thử
KT03	Lời mời không hợp lệ, C06	Từ chối tên rỗng, tự mời và offline	Đạt các trường hợp đã thử
KT04	Chấp nhận/từ chối lời mời, C06	Tra tên trim/khác hoa thường, tạo phòng, loại người mời khỏi hàng chờ	Đạt luồng đã thử
KT05	Ngắm liên tục, C03/G03	60 mẫu 34 byte được chuyển cho đối thủ, không echo về người gửi	Đạt relay; chưa kết luận độ mượt hai UI
KT06	Kết thúc mô phỏng, G07	PHYSICS_DONE chuyển lượt	Đạt chuyển lượt; không có chốt snapshot
KT07	Hết hạn/mất kết nối, C06–C07	Hết hạn khoảng 15 giây; phản hồi thất bại khi đối tác rời	Đạt các nhánh được kích hoạt
KT08	Form và refresh, UI02–UI03	Callback chạy đúng luồng ở các trường hợp đã thử	Có hành vi đúng, còn lỗi riêng bên dưới
KT09	Popup/toast, UI05	Popup trên owner, đóng theo vòng đời; kết quả dùng toast	Đạt trong fixture JavaFX

Mã	Trường hợp và nhóm UC	Quan sát thực tế	Đánh giá
KT10	Solo và nhận mẫu ngắm, UI06/G03/G08	Không còn khung đối thủ trống; đọc mẫu ngắm hiện hành	Đạt kiểm tra UI có kiểm soát

Lỗi và giới hạn cần hoàn thiện

Mã	Hiện tượng	Tác động	Thành phần cần hoàn thiện
L01	Hai READY từ cùng người có thể mở trận	Trận bắt đầu trước khi đối thủ thực sự sẵn sàng	Giao thức và trạng thái trận đấu
L02	Góc NaN được relay; cú đánh lặp tiếp tục được chuyển	Trạng thái bất hợp lệ hoặc áp dụng cú đánh nhiều lần	Giao thức và trạng thái trận đấu
L03	Phản hồi sai tên hủy timer nhưng còn currentChallenger	Lời mời có thể giữ trạng thái bận quá hạn	Vòng đời lời mời và phản hồi giao diện
L04	SETTINGS_RESPONSE server chỉ có boolean, client đọc thêm string	Luồng đọc không khớp; có thể chờ hoặc lỗi	Thông điệp, dữ liệu và phản hồi giao diện
L05	Email sai định dạng được server nhận; UI từ chối TLD dài hợp lệ	Quy tắc hai đầu không thống nhất	Kiểm tra dữ liệu và giao diện
L06	Hai tin giống nhau trong cùng phút bị gộp; lịch sử cũ vào cuộc chat mới	Mất biểu diễn tin hoặc sai hội thoại	Thông điệp, dữ liệu và phản hồi giao diện
L07	Refresh A→B→A khi menu mở áp dụng B lúc đóng	Danh sách hiển thị dữ liệu cũ	Cập nhật giao diện

Các dòng trên là hiện tượng đã ghi trong log. Ngoài ra, đọc mã cho thấy ghi kết quả chưa có transaction toàn vẹn, DB dùng Connection chung, mô phỏng chưa có bước thời gian cố định và chưa có snapshot/ACK hai phía. Những nhận xét từ mã cần được tách khỏi khẳng định đã gây lỗi ở mọi điều kiện thực tế.

Kế hoạch nghiệm thu tích hợp còn lại

Mã	Kịch bản cần chạy	Tiêu chí quan sát mong đợi	Trạng thái
NT01	Hai máy Windows dùng EXE cùng bản	Mở được, đủ tài nguyên, kết nối và đăng nhập thành công	Chưa thực thi trong bộ minh chứng
NT02	Ngắm liên tục, đổi góc qua biên ±π, kéo lực và chỉnh xoáy qua nhiều lượt	Hướng gậy đúng người có lượt; không kẹt hình ngắm sau đánh hoặc đổi lượt	Chưa nghiệm thu hai UI qua mạng

Mã	Kịch bản cần chạy	Tiêu chí quan sát mong đợi	Trạng thái
NT03	Chơi trọn ván với phạm lỗi, hết giờ, bi cái và bi 8 vào lỗ	Quyền lượt, kết quả và dữ liệu sau trận thống nhất với luật công bố	Chưa có ván đầy đủ trong bộ minh chứng
NT04	Thêm độ trễ/jitter, ngắt mạng giữa ngắm/đánh/lời mời	Có phản hồi và dọn phiên; không áp dụng lệnh của trạng thái cũ	Cần chạy sau khi sửa lỗi trạng thái
NT05	Gây lỗi DB giữa ghi kết quả, đăng ký đồng thời	Không ghi kết quả một phần; định danh tài khoản nhất quán	Chưa kiểm thử giao dịch/đồng thời đầy đủ
NT06	Lưu và đăng nhập lại với avatar/skin; thử âm thanh/mute	Quyền theo level đúng; tùy chọn tồn tại theo thiết kế	Chưa nghiệm thu toàn bộ tùy chọn
NT07	Nhiều phòng, lời mời chéo và chuyển chat nhanh	Phòng độc lập; phản hồi thuộc đúng đối tượng; đo tải có cấu hình rõ	Chưa có stress test/benchmark

Mỗi lần chạy cần lưu phiên bản mã và binary, hệ điều hành, cấu hình mạng, dữ liệu đầu vào, bước thao tác, kết quả mong đợi và thực tế. Với NT02–NT04, nên ghi cả thời điểm gửi/nhận và ảnh hai phía để phân biệt chậm hiển thị với sai trạng thái. Các tiêu chí này là kế hoạch kiểm tra, không được ghi là đã đạt chỉ vì tài liệu hoặc sơ đồ đã hoàn thành.

Đánh giá kết quả của nhóm

Dự án đã liên kết được các nhóm chức năng từ kết nối, tài khoản và sảnh đến bàn chơi, kết quả và cá nhân hóa trong một kiến trúc client–server. Các kiểm tra đã có cung cấp bằng chứng cho việc biên dịch, nhiều nhánh thách đấu, relay mẫu ngắm và một số phản hồi JavaFX. Việc chia bốn phân hệ giúp xác định nơi xử lý khi xuất hiện lỗi xuyên lớp, chẳng hạn sai payload nằm giữa giao thức, dữ liệu và giao diện.

Mức hoàn thiện còn giới hạn ở tính nhất quán trong các tình huống lặp, muộn hoặc sai trạng thái và ở phạm vi kiểm chứng trên Windows. Vì vậy, kết quả hiện tại phù hợp để trình bày sản phẩm đồ án cùng phân tích kỹ thuật và hướng hoàn thiện; chưa đủ cơ sở tuyên bố không còn lỗi bất đồng bộ hoặc bảo đảm hiệu năng trong mọi điều kiện mạng.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Đồ án xây dựng ứng dụng bi-a hai người qua mạng bằng Java/JavaFX, Socket và PostgreSQL, kết hợp nghiệp vụ tài khoản, tương tác xã hội, thi đấu và cá nhân hóa. Báo cáo tổng hợp toàn bộ phạm vi của nhóm, từ mục tiêu, kiến thức nền và kiến trúc đến đặc tả, triển khai, minh họa sản phẩm và đánh giá.

Hệ thống kết hợp kết nối và giao thức, xác thực và dữ liệu, gameplay và vật lý, giao diện và cá nhân hóa thành một quy trình sử dụng liên tục. Tổng cộng 31 đặc tả và 14 biểu đồ UML được đối chiếu với mã nguồn. Phần triển khai và kiểm thử đánh giá sự phối hợp giữa các thành phần, từ thao tác của người chơi đến xử lý tại server, lưu trữ và phản hồi trên màn hình.

Bài học kỹ thuật chính là giao thức cần được kiểm chứng đồng thời ở hai đầu, thao tác UI cần tuân thủ luồng JavaFX, và vận chuyển tin cậy chưa đủ để bảo đảm trạng thái trò chơi nhất quán. Các lỗi được ghi lại cung cấp đầu việc cụ thể cho giai đoạn hoàn thiện, thay vì suy luận chất lượng chỉ từ giao diện hoặc việc biên dịch thành công.

Hướng phát triển theo mức ưu tiên

Ưu tiên trước hết là sửa những sai lệch đã tái hiện: thống nhất SETTINGS_RESPONSE; chống READY và cú đánh lặp; kiểm tra giá trị hữu hạn/phạm vi tham số; bảo vệ vòng đời lời mời; gắn phản hồi chat với đúng hội thoại và thống nhất kiểm tra email. Sau mỗi thay đổi cần tái lập trường hợp lỗi rồi chạy lại luồng bình thường liên quan.

Bước tiếp theo là củng cố trạng thái trận bằng mã trận/lượt, thứ tự lệnh và quy tắc chấp nhận thông điệp. Có thể lựa chọn server mô phỏng có thẩm quyền hoặc kết hợp bước thời gian cố định ở client với cơ chế chốt/hiệu chỉnh bàn. Thiết kế được chọn cần được đánh giá bằng hai client thật, đo sai lệch vị trí và kiểm tra mất kết nối, không chỉ bổ sung tên packet vào tài liệu.

Về dữ liệu, cần transaction cho kết quả trận, quản lý kết nối phù hợp tải đồng thời và ràng buộc định danh nhất quán với cách tra cứu. Về triển khai, cần cấu hình ngoài mã, gói Windows có mã phiên bản rõ, kiểm tra trên máy sạch và quy trình phát hành riêng với source ZIP. TLS và thuật toán băm mật khẩu chuyên dụng cũng cần được xem xét trước khi mở rộng sử dụng.

Sau khi nền tảng ổn định, nhóm có thể bổ sung tái kết nối, lịch sử trận chi tiết hoặc tính năng quan sát. Những chức năng này là định hướng tương lai và không nằm trong 31 use case hiện tại.

PHỤ LỤC BÀN GIAO VÀ THUẬT NGỮ

Thành phần bàn giao

Nhóm tài liệu/mã	Vị trí	Mục đích
Báo cáo tổng	reports/ Bao_cao_tong_hop_nhom_LTM.*	Word để chỉnh sửa, PDF để đọc/in, Markdown/HTML để tra cứu
Báo cáo bốn phân hệ	reports/Bao_cao_LTM_4_phan.*	Tài liệu đặc tả kỹ thuật chi tiết
Đặc tả và biểu đồ	reports/uml_spec.json, uml_manifest.json, uml/	31 UC; 14 nguồn PlantUML và ảnh PNG/SVG
Minh họa UI	reports/images/	Ảnh thật có thông tin nguồn và giới hạn kiểm chứng
Minh chứng	reports/evidence/00bf1cc/	Script, README và log ngày 26/10/2026
Mã triển khai	client/, server/, database/	Ứng dụng, script build và schema

Thông tin hành chính cần hoàn thiện trước khi nộp gồm trường/khoa/lớp, giảng viên, tên nhóm, họ tên đầy đủ và MSSV của các thành viên. Những trường còn thiếu được để nhãn bổ sung. Nếu trường có mẫu bìa hoặc quy định riêng, có thể điều chỉnh trong Word và giữ nguyên cấu trúc chuyên môn của báo cáo.

Thuật ngữ

Thuật ngữ	Ý nghĩa trong báo cáo
UC	Use case, trường hợp sử dụng/chức năng trong phạm vi phân hệ
UML	Ngôn ngữ mô hình hóa; báo cáo dùng Use Case, Activity, Sequence và Component
RTT	Thời gian khứ hồi đo bằng PING và phản hồi
Payload	Các trường dữ liệu theo sau mã loại packet
HUD	Thông tin hiển thị trên màn hình chơi: người chơi, lượt và trạng thái
Fixture	Dữ liệu/transport có kiểm soát phục vụ kiểm tra, không phải kết nối đầu cuối thực
Snapshot/ACK	Ảnh chụp trạng thái và xác nhận; hướng hoàn thiện, chưa có trong giao thức hiện tại
EXE app-image	Launcher và runtime/tài nguyên đi kèm do jpackage đóng gói trên Windows